МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования
**«Национальный исследовательский
Нижегородский государственный университет им. Н.И. Лобачевского»
(ННГУ)**

Институт информационных технологий, математики и механики

Кафедра: Математического обеспечения и суперкомпьютерных технологий

Направление подготовки: «Прикладная математика и информатика»
Профиль подготовки: «Общий профиль»

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

на тему:

«Программная реализация и исследование алгоритма поиска нуля многоэкстремальной функции»

Выполнила:
студентка группы 3821Б1ПМоп3

_____Шемякина А.П.
Подпись

Научный руководитель:
д.т.н., доц., зав. кафедрой МОСТ института ИТММ

_____Баркалов К.А.
Подпись

Нижний Новгород
2025

Оглавление

Введение 3
1. Постановка задачи 4
1.1 Виды задач оптимизации 4
1.2 Задачи многоэкстремальной оптимизации 5
2. Обзор методов решения задач оптимизации 6
3. Базовый алгоритм глобального поиска(АГП) 8
3.1 Вычислительная схема алгоритма 8
3.2 Пример работы алгоритма 9
4. Алгоритм с двойной оценкой константы Липшица(АДОК) 11
4.1 Вычислительная схема алгоритма 11
4.2 Пример работы алгоритма 12
5. Алгоритм поиска нуля(АПН) 13
5.1 Вычислительная схема алгоритма 13
5.2 Пример работы алгоритма 14
6. Результаты вычислительных экспериментов 16
6.1 Методика сравнения алгоритмов 16
6.2 Операционные характеристики алгоритмов 17
7. «Проблемные» функции для алгоритмов 25
Заключение 39
Литература 41
Приложение 42

Введение

Как во всей природе, так и в человеческой жизни, существует множество примеров оптимальности. В природе расположение листьев и семян на деревьях оптимально для того, чтобы на них попадало больше света, а, например, миграции животных производятся по оптимальным маршрутам, требующих минимальных энергозатрат. А люди стремятся к улучшению технологий и уровня жизни, но ищут оптимальные пути к этому, с меньшей тратой ресурсов. Такие задачи оптимизации возникают по причине того, что все в мире направлено на поиск гармонии и стремление к идеалу.

Для целей оптимизации наилучшим образом подходит математическая модель проектируемого объекта, так как проведение множества экспериментов с целью совершенствования будет менее затратным, чем если бы все проводилось на реальной конструкции. Следует отметить, что при решении сложных задач оптимизации ЭВМ играет ключевую роль, позволяя находить наилучшие решения в сложных системах с множеством параметров и ограничений.

В наше время уже изучено множество задач и методов их решения. Так, например, давно известно, как решать задачу поиска минимума некоторой непрерывной функции – найти ноль производной. В задачах экономического характера функции и их ограничения обычно являются линейными и данная область также детально проработана: известные методы линейного программирования успешно решают подобные задачи. Однако если функции имеют вид «черный ящик», то есть не заданы формулами, то такие хорошо проработанные, известные методы неприменимы. Это обусловлено тем, что многолетние подходы к решению задач поиска минимума, опираются на формульное задание. Когда известна формула, то есть возможность найти производную и далее отталкиваться от этого знания. Также и все методы локальной оптимизации опираются на понятие градиента функции, что, несомненно, требует знания формульного задания функции. Не подходят такие методы и к многоэкстремальным функциям. Дело в том, что с их помощью из заданной начальной точки можно спуститься в точку локального минимума, в которой, по определению, значение целевой функции меньше, чем все значения в соседних точках. Однако локальный минимум совпадает с глобальным далеко не всегда.

Целью данной работы является исследование алгоритмов глобальной оптимизации, применяемых для решения задач многоэкстремальной оптимизации. В рамках работы будут рассмотрены основные подходы к решению задач оптимизации, исследована применимость рассматриваемых алгоритмов к различным примерам одномерных функций с несколькими экстремумами. В числе исследуемых алгоритмов будут: алгоритм глобального поиска, алгоритм с двойной оценкой константы Липшица, а также новый метод – алгоритм поиска нуля. Будут проанализированы эффективность и сравнение операционных характеристик данных алгоритмов на примере работы с двумя функциональными семействами. С целью более глубокого исследования работы и поведения алгоритмов, будут выявлены самые сложные задачи семейств для каждого из методов.

1 Постановка задачи

Конкретная постановка задачи оптимального выбора связана с фиксацией части параметров, установкой ограничителей на переменные, оставшихся свободными, выделением групп оптимизируемых и ограничиваемых характеристик функционирования.

Все алгоритмы оптимизации ищут наилучшее решение среди многих других, для этой цели рассматриваемые варианты должны быть сравнимы: сопоставив каждому варианту число – критерий оптимальности, получим возможность поручить выбор наилучшего варианта ЭВМ.

В соответствии с [3], обозначим множество всех вариантов – D, а его элементы – X. Тогда, сопоставив каждому варианту X из множества D $(X \in D)$ число – критерий оптимальности, получим функцию $f(X)$, определенную в области D. Данная функция, показывающая «качество» выбираемых вариантов, называется целевой функцией. Область D – допустимая область.

Тогда задачу оптимизации(минимизации) можно записать следующим образом:

$$\min_{X \in D} f(x) \qquad (1)$$

Далее в основном будут упоминаться задачи минимизации, потому что задача максимизации целевой функции $f(x)$ равносильна минимизации, но с отрицательной целевой функцией.

Если целевых функций в задаче несколько, то мы имеем многокритериальную задачу оптимизации, а также векторный критерий оптимальности и векторную целевую функцию

$$f(X) = \big(f_1(X), \dots, f_s(X)\big),$$

состоящую из s компонентов.

1.1 Виды задач оптимизации

Классификацию задач определяют две составляющие – целевая функция $f(X)$ и допустимая область D.

Методы решения задач оптимизации особенно сильно зависят от принадлежности задачи к одному из двух классов:

Задача одноэкстремальной оптимизации. Такая задача принадлежит к классу одноэкстремальных(унимодальных) функций, которые имеют единственный локальный экстремум, который при этом является и глобальным.

Задача многоэкстремальной оптимизации. Такая задача принадлежит к классу многоэкстремальных(мультимодальных) функций, которые имеют несколько локальных экстремумов. Алгоритмы решения таких задач ведут поиск глобального оптимума среди множества локальных.

Такая классификация связана с существованием минимумов двух типов – локальных и глобальных. Для минимума в точке X^* справедливо неравенство

$$f(X^*) \leqslant f(X)$$

Разница между глобальным и локальным минимумами заключается в том, что для точки глобального минимума X^* неравенство справедливо для всех $X \in D$, а для точки локального минимума – только для точек некоторой окрестности минимума $d(X^*)$. Иначе говоря, локальный минимум является минимумом только среди его окружающих ближайших точек.

Следует отметить, что многоэкстремальность зависит в частности от допустимой области: уменьшив область, эту задачу можно превратить в одноэкстремальную. Очень важно выявить многоэкстремальна задача оптимизации или нет, потому что от этого существенно зависит выбор метода, а также трудности на пути к её решению.

Допустимую область D определяют системы неравенств или равенств, иногда специальные алгоритмы. Их называют ограничениями задачи оптимизации. Различают линейные ограничения, дифференцируемые нелинейные, недифференцируемые нелинейные (см. [3]).

1.2 Задачи многоэкстремальной оптимизации

В случае, когда задаче (1) соответствует несколько точек локального минимума, задача называется многоэкстремальной. При этом точка X^* искомого решения называется точкой глобального или абсолютного минимума. Многоэкстремальность задачи является следствием нелинейной(и невыпуклой) зависимости некоторых(или всех) функций f и $g_i, 1 \leqslant i \leqslant m$(ограничения-неравенства области D) от вектора x.

Всякая надежная оценка глобального оптимума в многоэкстремальной задаче опирается на наличие некоторой априорной(имеющейся до начала вычислений) информации о решаемой задаче. Данная информация может быть получена, исходя из физической сущности задачи, к примеру, сюда относятся её непрерывность, гладкость, монотонность. Для удобного рассмотрения и большей наглядности предположим, что допустимая область в задаче совпадает с областью поиска, иначе говоря, ограничения отсутствуют. Воспользуемся тем фактом, что в большом числе прикладных задач изменение вектора параметров x вызывает изменение характеристик объекта, ограниченное некоторой мерой изменения x.

Модель, описывающая свойство ограниченности изменений, для рассматриваемой задачи с единственной характеристикой $f(x)$, может выглядеть так:

$$|f(x') - f(x'')| \leqslant L\rho(x', x''), x', x'' \in D,$$

где $\rho(x', x'')$ – некоторая функция расстояния в пространстве параметров, а L – заданная константа. В случае, когда

$$\rho(x', x'') = (\|x' - x''\|)^{\frac{1}{N}},$$

где функция $\|x' - x''\|$ соответствует евклидовой метрике, говорят, что функция f удовлетворяет равномерному условию Гельдера с показателем $\frac{1}{N}$ и коэффициентом L. При $N = 1$ это условие называется условием Липшица с константой L (см. [5]).

2 Обзор методов решения задач оптимизации

Дана многоэкстремальная задача: пусть $f(x)$ – действительная функция, определенная в гиперкубе D N-мерного евклидова пространства R^N:

$$D = \{x \in R^N : a_i \leqslant x_i \leqslant b_i, 1 \leqslant i \leqslant N\}, \quad (2)$$

где $a, b \in R^N$ – заданные векторы, и пусть в точке x^* функция $f(x)$ достигает минимального значения на множестве D:

$$f(x^*) = \min_{x \in D} f(x) \quad (3)$$

Требуется построить оценку $x_* \in D$ точки x^* на основе конечного числа k значений функции, последовательно вычисленных в выбранных точках области D. Оценка строится для некоторого принятого понятия близости, к примеру, $\|x^* - x_*\| \leq \varepsilon$, где ε – заданная точность.

Будем говорить, что операция вычисления характеристик функции в точке – поисковое испытание, а совокупность значений характеристик в этой точке – результат испытания. Далее результатом испытания будет называться значение функции в испытуемой точке.

Результатом испытания в точке $x^i \in D$ в общем случае будет число $z^i = f(x^i) + g^i$, где g^i – погрешность i-го испытания. Также стоит отметить, что последовательное проведение испытаний подразумевает возможность выбора точки x^i с учетом результатов $z^1, \dots, z^{i-1}$ предыдущих испытаний. Приведем краткое описание известных алгоритмов оптимизации в соответствии с [6].

0) Метод последовательного сканирования(перебор).

Алгоритм перебора – самый простейший алгоритм для решения задачи многоэкстремальной оптимизации. Для данного метода характеристика интервала – это его длина. После чего осуществляется поиск интервала, которому соответствует максимальная характеристика, а точка очередного испытания выбирается в середине этого интервала.

1) Области притяжения и выбор начальных точек.

Пусть в некоторой подобласти D_i, $1 \leqslant i \leqslant m$, области D (2), функция $f(\mathrm{x})$ (3) является унимодальной, где x_i^* – точка локального минимума функции $f(\mathrm{x})$ в подобласти D.

Тогда, применяя методы локального спуска, для любой начальной точки $\mathrm{x_i}$, лежащей в подобласти D_i, можно вычислить соответствующую точку локального минимума $x_i^* \in D$. Отсюда говорят, что подобласть D является областью притяжения локального минимума $\mathrm{x_i}$. Таким образом, решение данной многоэкстремальной задачи (2), (3) может быть сведено к решению некоторого числа m задач на локальный экстремум, если задано множество начальных точек $x_i^0 \in D_i, 1 \leq i \leq m$. Однако данное требование для некоторых задач может быть сильным, поэтому при таком подходе возникает дополнительная задача выбора начальных точек. К примеру, используется выбор начальных точек градиентного спуска по схеме метода Монте-Карло или чередование итераций локального спуска с выбором новых начальных точек.

2) Случайный поиск и автоматный подход.

В алгоритмах для решения многоэкстремальных задач, входящих в класс методов случайного поиска, отсутствуют какие-либо итерации локального спуска. А точка x^{t+1} очередного испытания выбирается согласно выражению:

$$x^{t+1} = x^t + h^t,$$

где h^t – реализация N-мерной векторной случайной величины, распределение вероятностей которой определяется конкретным типом алгоритма случайного поиска. Один из способов описания таких алгоритмов связан с использованием языка теории вероятностных автоматов. Последовательность $\{x^t\}$ можно рассматривать как реализацию случайного процесса, которая при возрастании t должна пересекаться с заданной окрестностью точки абсолютного минимума.

3) Многоэкстремальная стохастическая аппроксимация.

В данной схеме осуществляется совмещение операций выбора начальной точки и локального спуска. Смещение точки при переходе от одного испытания к другому включает компоненту, пропорциональную антиградиенту функции, а также компоненту, являющуюся реализацией скачкообразного случайного процесса. Достижение асимптотической сходимости к абсолютному экстремуму обеспечивается путем соответствующего управления длиной градиентного шага и интенсивностью случайных толчков.

4) Применение сглаживания и фильтрации.

Проведение испытаний в точках выборки с последующей оценкой среднего значения результатов позволяет перейти от минимизации исходной функции $f(x)$ к минимизации некоторого усреднения этой функции. Это приводит к построению интегрального преобразования $\Phi(x)$ – сглаженной версии $f(x)$, обладающей тем свойством, что её минимальное значение совпадает с минимальным значением исходной функции. При этом $\Phi(x)$ является одноэкстремальной функцией, что делает возможным применение локальных методов оптимизации для ее минимизации.

5) Использование аналогий с физическими системами.

Некоторые алгоритмы для решения многоэкстремальных задач используют аналогию с механической системой, представляя минимизируемую функцию в виде некоторой поверхности, по которой движется тяжелый шарик. Такая аналогия содержит потенциальное поле, в котором происходит движение, а минимизируемая функция описывает потенциал этого поля. Существуют также аналогия с моделями статистической физики, предложенная для решения систем нелинейных алгебраических уравнений. Данный метод может быть использован также и в многоэкстремальных задачах. А, к примеру, аналогия методов случайного поиска и гидродинамической модели, служит для установления вариационных принципов случайного поиска.

Ознакомившись с методами решения многоэкстремальных задач оптимизации, возьмем на рассмотрение несколько алгоритмов глобальной оптимизации, реализуем программно, исследуем их работу и выявим эффективность.

3 Базовый алгоритм глобального поиска(АГП)

Данный алгоритм предназначен для отыскания точек x^*и значений f^* глобального минимума действительной функции $f(x)$ на отрезке $[a,b]$ вещественной оси. Запишем её так:

$$f^* = f(x^*) = \min_{x \in D=[a,b]} f(x) \tag{4}$$

Согласно [6], любой алгоритм, предназначенный для решения данной задачи (4), порождает последовательность точек x^k ($k = 0, \ldots$), в них вычисляются значения минимизируемой функции – z^k, причем $z^k = f(x^k)$.

Для описания алгоритма, зададим функции, аргументом которых является множество пар

$$w = w_k = (x_i, z_i): 1 \le i \le k, \tag{5}$$

соответствующих первым k членам последовательности $\{x^k\}$, называемой, в соответствии с [6], последовательностью испытаний.

Предполагается, что данные рассматриваемые функции $f(x)$ удовлетворяют условиям Липшица, то есть должны выполняться неравенства:

$$|f(x_1) - f(x_2)| \leqslant L|x_1 - x_2|,$$

для любых $x_1, x_2 \in [a,b]$,

где L – константа Липшица. Выполнимость условия Липшица означает ограниченность вариации значения функции при изменении значения ее параметра, что позволяет устанавливать гарантированные оценки снизу для значений минимизируемой функции в точках области поиска глобального минимума по результатам проведенных итераций.

3.1 Вычислительная схема алгоритма

Пусть множество (5) будет называться «поисковая информация», а испытание, проводимое при наличии информации w_k – $(k+1)$-м испытанием.

Согласно непрерывному алгоритму(см. [6]) первые два испытания осуществляются в точках $x^0 = a, x^1 = b$. Выбор точки $x^{k+1}, k \geqslant 2$, очередного $(k+1)$ -го испытания определяется следующими правилами:

1) перенумеровать (нижним индексом) точки x^i в порядке возрастания значений координаты: $a = x_0 < x_1 < \ldots < x_{k-1} = b$;

2) оценить максимальное абсолютное значение относительной первой разности

$$M = \max_{1 \leqslant i \leqslant k-1} \left| \frac{z_i - z_{i-1}}{x_i - x_{i-1}} \right|,$$

где $z = f(x_i), 1 \leqslant i \leqslant k$ соответствуют перенумерованным значениям величин z^i;

3) положить

$$m = \begin{cases} 1, M = 0 \\ M, M > 0 \end{cases},$$

4) для каждого интервала $(x_{i-1}, x_i), 1 \leqslant i \leqslant k-1$, вычислить величину

$$R(i) = (x_i - x_{i-1}) + \frac{(z_i - z_{i-1})^2}{m^2 r^2 (x_i - x_{i-1})} - 2\frac{z_i + z_{i-1} - 2z^*}{mr},$$

называемую характеристикой этого интервала; z^* вычисляется по формуле:

$$z^* = \min_{0 \le i \le k-1} z_i$$

5) найти интервал (x_{t-1}, x_t), которому соответствует максимальная характеристика

$$R(t) = \max_{1 \leqslant i \leqslant k-1} R(i)$$

В случае нескольких интервалов характеристики, которые удовлетворяют данному выражению, выбирается интервал с наименьшим номером t.

6) провести новое испытание в точке

$$x^{k+1} = \frac{x_t + x_{t-1}}{2} - \frac{z_t - z_{t-1}}{2mr},$$

вычислить значение $z^{k+1} = f(x^{k+1})$ и увеличить номер шага на единицу: $k = k + 1$

Данные правила описывают некоторую последовательность отображений

$$x_{k+1} = G_k^r(x^0, \dots, x^k; z^0, \dots, z^k), k = 1,2, \dots,$$

определяющих точки и обладающих свойством инвариантности

$$G_k^r(x^0, \dots, x^k; z^0, \dots, z^k) = G_k^r(x^0, \dots, x^k; \alpha z^0 + \beta, \dots, \beta z^k + \beta),$$

где $\alpha > 0$ и β –вещественные числа.

Условие остановки в данном алгоритме определяется правилом:

$$x_t - x_{t-1} \leqslant \varepsilon$$

В качестве оценки экстремума выбирается пара $\mathrm{e}^k = (f_k^*, x_k^*)$

где f_k^* – минимальное вычисленное значение функции: $f_k^* = \min_{1 \leqslant i \leqslant k} z^i$,

а x_k^* – координата этого значения: $x_k^* = arg \min_{1 \leqslant i \leqslant k} f(x^i)$

3.2 Пример работы алгоритма

Проверим работу алгоритма глобального поиска на некоторой многоэкстремальной одномерной тестовой задаче:

$$f(x) = x^2 - \cos 18x + \sin 50x + 1.878638$$

Перед запуском программы[1] зададим необходимые параметры:

- Точность $\varepsilon = 0.0001$;
- Область поиска в виде интервала $x \in [a, b] = [-0.5, 1.5]$;
- Параметр метода $r = 2$;
- Максимальное число итераций $Kmax = 1000$;

[1]код программы с реализацией функций приведен в приложении.

Остановка производится по точности.

Результат работы программы по поиску глобального минимума данной задачи:
Текущий результат:

$$z = 1.84211e - 06$$
$$x = 0.345772$$
$$k = 111$$

Лучший результат:

$$z = 1.75626e - 06$$
$$x = 0.345689$$
$$k = 81$$

Здесь под «Текущим результатом» понимается испытание, во время которого метод остановился. «Лучший результат» – испытание, которое метод запомнил как наилучший минимум функции. z – значение функции, x – аргумент функции, k – номер испытания.

На представленном далее графике(Рисунок 1) можно увидеть, в каких местах метод ставил очередное испытание(первый ряд штрихов под функцией), предполагая, что в данной точке может находиться глобальный минимум. Также заметны зоны сгущения – места, где алгоритм проводил испытания чаще. В соответствии с графиком, это окрестности точек подозрительных на минимум соответственно.

Можно также отметить, что самые большие зоны сгущения находятся в двух точках с очень похожими значениями функций. Обе могут претендовать на глобальный минимум функции. По результатам программы, метод обнаружил его в точке $x = 0.345689$ за 111 итераций.

4 Алгоритм с двойной оценкой константы Липшица(АДОК)

Представленный выше алгоритм глобального поиска предназначен для решения многоэкстремальных задач, в которых целевая функция удовлетворяет условию Липшица. Однако необязательно определять значение константы Липшица во время работы алгоритма, так как оценка константы уже производится в ходе глобального поиска на основе имеющейся поисковой информации. В соответствии с [8], следует сказать, что по теореме о достаточном условии сходимости в [7], точка глобального минимума x^* является предельной точкой последовательности $\{x^k\}$, если выполняется условие:

$$rm > 2^{3-1/N} L\sqrt{N+3} \tag{6}$$

Тогда величина $(rm)/2^{3-1/N}\sqrt{N+3}$, где r – выбранный параметр из пункта 4), может использоваться как оценка константы Липшица для целевой функции $f(x)$.

Условие (6) будет гарантированно выполнено в случае, если параметр r будет достаточно большим. Однако тогда метод будет выполнять большое количество испытаний до момента срабатывания условия остановки. Если же выберем маленькое значение r, это уменьшит число испытаний, но может нарушить сходимость к глобальному экстремуму. Тогда было принято решение при определенных условиях использовать параметр r_{loc}(что соответствует нижней оценке константы Липшица), а в других случаях r_{glob}(соответствует большей оценке константы), с целью корректно улучшить работу алгоритма. $r_{glob} > r_{loc} > 1$.

4.1 Вычислительная схема алгоритма

Правила выбора точки в алгоритме с двойной оценкой константы Липшица во многом повторяют правила алгоритма глобального поиска, за исключением пункта 4) – вычисление характеристики, и пункта 5) – поиск интервала с максимальной характеристикой. Рассмотрим данные изменения в правилах.

Исходя из [8], вычисление характеристики производится следующим образом:

– Значение $R_{glob}(i)$, соответствующее наибольшей оценке константы Липшица

$$R_{glob}(i) = (x_i - x_{i-1}) + \frac{(z_i - z_{i-1})^2}{m^2 r_{glob}^2 (x_i - x_{i-1})} - 2\frac{z_i + z_{i-1} - 2z^*}{m r_{glob}}$$

– Значение $R_{loc}(i)$, соответствующее наименьшей оценке константы Липшица

$$R_{loc}(i) = (x_i - x_{i-1}) + \frac{(z_i - z_{i-1})^2}{m^2 r_{loc}^2 (x_i - x_{i-1})} - 2\frac{z_i + z_{i-1} - 2z^*}{m r_{loc}}$$

– Определяем характеристику $R(i)$ как

$$R(i) = \max_i \{\rho R_{loc}(i), R_{glob}(i)\}, \text{где } \rho = \left(\frac{1 - 1/r_{glob}}{1 - 1/r_{loc}}\right)^2$$

Далее поиск интервала (x_{t-1}, x_t), которому соответствует максимальная характеристика:

$$R(t) = \max_{1 \leqslant i \leqslant k-1} R(i)$$

А также, фиксируем значение $r = r_{loc}$, если $\rho R_{loc}(t) > R_{glob}(t)$, иначе $r = r_{glob}$. Далее используем это значение r в пункте 6) алгоритма глобального поиска(проведение нового испытания в точке x^{k+1}).

4.2 Пример работы алгоритма

Проверим работу алгоритма с двойной оценкой константы Липшица на некоторой многоэкстремальной одномерной тестовой задаче:

$$f(x) = x^2 - \cos 18x + \sin 50x + 1.878638$$

Перед запуском программы[2] зададим необходимые параметры:

- Точность $\varepsilon = 0.0001$;
- Область поиска в виде интервала $x \in [a, b] = [-0.5, 1.5]$;
- Параметры метода $r_{loc} = 1.3, r_{glob} = 3.6$;
- Максимальное число итераций $Kmax = 1000$;

Остановка производится по точности.

Результат работы программы по поиску глобального минимума данной задачи:
Текущий результат:

$$z = 2.3082e - 06$$
$$x = 0.345685$$
$$k = 140$$

Лучший результат:

$$z = 1.17763e - 06$$
$$x = 0.345766$$
$$k = 98$$

z – значение функции, x – аргумент функции, k – номер испытания.

На представленном далее графике(Рисунок 1) можно увидеть, в каких местах метод ставил очередное испытание(третий ряд штрихов под функцией), предполагая, что в данной точке может находиться глобальный минимум. Также заметны зоны сгущения – места, где алгоритм проводил испытания чаще. В соответствии с графиком, это окрестности точек подозрительных на минимум.

Можно также отметить, что самые большие зоны сгущения находятся в двух точках с очень похожими значениями функций. Обе могут претендовать на глобальный минимум

[2]код программы с реализацией функций приведен в приложении.

функции. По результатам программы, метод обнаружил его в точке $x = 0.345766$ за 140 итераций.

5 Алгоритм поиска нуля(АПН)

Рассмотрим одномерную задачу минимизации функции на отрезке

$$\varphi(x) \to min, x \in D = [a, b]$$

Отметим, что $\varphi(x) \geq 0$.

Алгоритм поиска нуля состоит в том, что на каждом шаге очередное испытание осуществляется в точке $x = \alpha^*$, являющейся оценкой нуля. Проще говоря, метод осуществляет поиск нуля функции, заведомо полагая, что там находится глобальный минимум. Следовательно, нам известно, что $\varphi(\alpha^*) = 0$. Задача состоит в том, чтобы найти α^*.

5.1 Вычислительная схема алгоритма

Первые два испытания осуществляются в точках $x^0 = a, x^1 = b$. Выбор следующей точки $x^{k+1}, k \geqslant 2$ определяется следующими правилами:

1) перенумеровать (нижним индексом) точки x^i в порядке возрастания значений координаты: $a = x_0 < x_1 < \ldots < x_{k-1} = b$;

2) для каждого интервала $(x_{i-1}, x_i), 1 \leqslant i \leqslant k-1$, вычислить величину

$$R(i) = \frac{z_i + z_{i-1}}{x_i - x_{i-1}}$$

называемую характеристикой этого интервала;

3) найти интервал (x_{t-1}, x_t), которому соответствует минимальная характеристика

$$R(t) = \min_{1 \leqslant i \leqslant k-1} R(i)$$

В случае нескольких интервалов характеристики, которые удовлетворяют данному выражению, выбирается интервал с наименьшим номером t.

4) провести новое испытание в точке

$$x^{k+1} = \frac{x_t + x_{t-1}}{2} - \left(\frac{z_t - z_{t-1}}{z_t + z_{t-1}}\right)(x_t - x_{t-1})\frac{1}{2r},$$

вычислить значение $z^{k+1} = f(x^{k+1})$ и увеличить номер шага на единицу: $k = k + 1$.

В качестве оценки нуля, соответствующей шагу k, примем точку $x_k^* = x_\alpha$,

$\alpha = arg \min_{1 \leqslant i \leqslant k} z(x^i), \varphi_k^* = \min_{1 \leqslant i \leqslant k} z(x^i)$.

Условие остановки в данном алгоритме определяется правилом: $\varphi(x_t) \leq \varepsilon$.

5.2 Пример работы алгоритма

Проверим работу алгоритма поиска нуля на некоторой многоэкстремальной одномерной тестовой задаче:

$$f(x) = x^2 - \cos 18x + \sin 50x + 1.878638$$

Перед запуском программы[3] зададим необходимые параметры:

- Точность $\varepsilon = 0.0001$;
- Область поиска в виде интервала $x \in [a, b] = [-0.5, 1.5]$;
- Параметр метода $r = 2$;
- Максимальное число итераций $Kmax = 1000$;

Остановка производится по точности.

Результат работы программы по поиску глобального минимума данной задачи:
Текущий результат:

$$z = 1.07654e - 06$$
$$x = 0.345696$$
$$k = 26$$

Лучший результат:

$$z = -6.32918e - 07$$
$$x = 0.345731$$
$$k = 25$$

z – значение функции, x – аргумент функции, k – номер испытания.

На представленном графике(Рисунок 1) можно увидеть, в каких местах метод ставил очередное испытание(второй ряд штрихов под функцией), предполагая, что в данной точке может находиться ноль функции. Также заметны зоны сгущения – места, где алгоритм проводил испытания чаще. В соответствии с графиком, это окрестности точек подозрительных на минимум.

В данной функции, как уже было упомянуто раньше, есть сложность – два значения функции очень похожи и оба могут претендовать на глобальный минимум. Стоит отметить, что алгоритм быстро определил, в какую из двух подозрительных окрестностей будет ставить больше точек. Обнаружение нуля функции(глобального минимума) произошло спустя 26 испытаний в точке $x = 0.345731$.

[3]код программы с реализацией функций приведен в приложении.

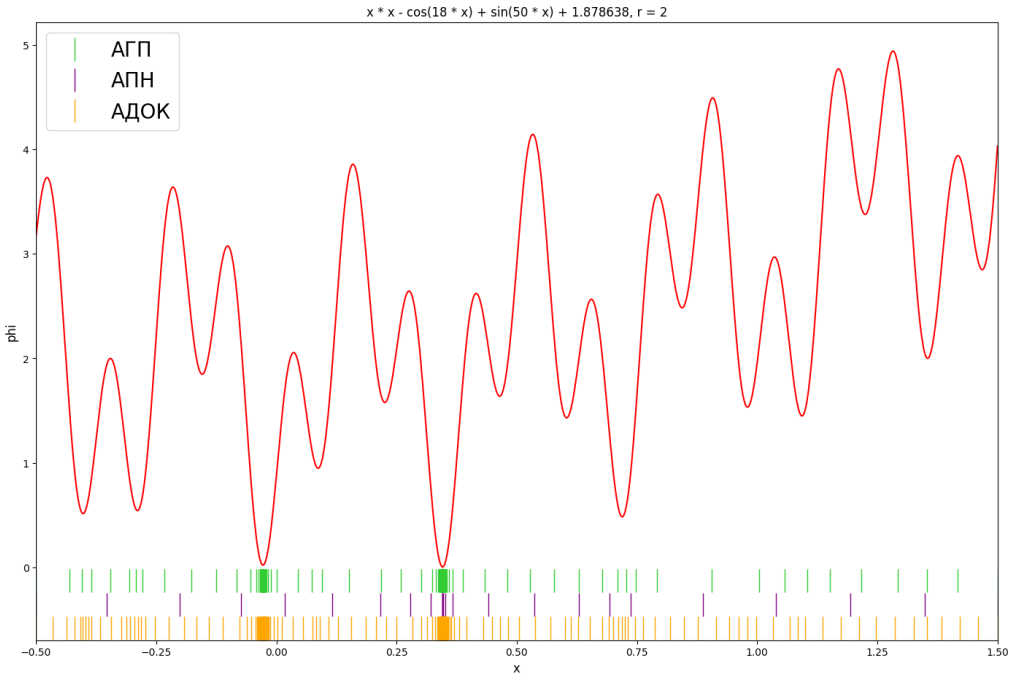


Рисунок 1 – Результат работы АГП, АДОК, АПН

6 Результаты вычислительных экспериментов

В данном разделе будут представлены качественные и количественные выкладки с результатами вычислительных экспериментов каждого из трех алгоритмов: АГП(базовый алгоритм глобального поиска), АДОК(алгоритм с двойной оценкой константы Липшица), АПН(алгоритм поиска нуля). Посредством данных выкладок будет произведено сравнение алгоритмов.

6.1 Методика сравнения алгоритмов

Как сказано в [9], тестовые функции – это искусственные проблемы, которые могут использоваться для оценки поведения алгоритмов в порой разнообразных и сложных ситуациях. Такие проблемы можно подвергать различным изменениям для проверки работы алгоритмов при различных сценариях. Как известно, функции больше чем с одним локальным минимумом называются мультимодальными. Такие функции удобно использовать для проверки возможности алгоритмов выходить за пределы любого локального минимума. Если процесс исследования алгоритма плохо спроектирован, то он не сможет эффективно исследовать ландшафт функции. Данное поведение может привести к тому, что алгоритм застрянет на локальном минимуме.

Итак, для сравнения операционных характеристик и эффективности рассматриваемых алгоритмов, воспользуемся случайной генерацией многоэкстремальных функций, которые будут формироваться в результате случайного выбора среди тестовых задач двух функциональных семейств.

Функции Шекеля – это семейство мультимодальных тестовых функций, которые широко используются как инструмент для проверки эффективности алгоритмов глобальной оптимизации. Функции были предложены Дж. Шекелем в 1971 году в докладе "Test Functions for Multimodal Search Techniques" на пятой принстонской конференции по информатике и информационным системам ([1]). Эти функции обеспечивают разнообразие экстремумов, что дает возможность эффективно тестировать алгоритмы, направленные на решение сложных задач оптимизации. Общий вид функции Шекеля для одномерной задачи оптимизации может иметь вид (см. [4]):

$$f(x) = -\sum_{i=1}^{10} \frac{1}{(K_i(x - A_i)^2 + C_i)}$$

при $0 \leqslant x \leqslant 10$. Это функция с 10-ю минимумами, где коэффициент A_i – координата минимума на оси x, C_i – глубина минимума, K_i– крутизна минимума(чем больше, тем уже и резче воронка).

Семейство функций Хилла и Гибсона – это набор математических функций, используемых для аппроксимации или моделирования сложных данных, например в контексте обработки сигналов или физических моделей. Эти функции являются мультимодальными, с большим количеством локальных минимумов. Соответственно, с их

помощью также можно оценить эффективность работы алгоритмов глобальной оптимизации. Функция Хилла и Гибсона определяется следующим выражением (см. [4]):

$$f(x) = A_0 + \sum_{i=1}^{14} \left(A_i sin(2i\pi x) + B_i cos(2i\pi x) \right)$$

при $0 \leqslant x \leqslant 1$.

Определение значений коэффициентов в обеих формулах реализуется при помощи генератора псевдослучайных чисел. Данные функции являются тестовыми, поэтому точные решения известны и включены в программу. Такой подход позволяет отслеживать сходимость найденного решения к точному.

6.2 Операционные характеристики алгоритмов

Для построения операционных характеристик алгоритмов, воспользуемся таким механизмом: отследим, при каком проценте от всех функций конкретного семейства, рассматриваемый алгоритм смог достичь точного решения спустя $10, 20, 30, \dots, 160, \dots$ итераций. Также выясним, за какое минимальное число итераций, во всех функциях семейства, алгоритм способен найти точные решения. Заметим, что в обоих семействах по 1000 функций.

Следует отметить, что было произведено поднятие над осью абсцисс всех функций обоих семейств, причем так, что глобальный минимум функции оказывался в её нуле. Это было сделано с целью сравнения трех рассматриваемых алгоритмов, в число которых входит АПН(Алгоритм поиска нуля). Заметим, что так как все функции тестовые, то нам известны их точные решения. Функции поднимались с помощью разности значений функции в каждой ее точке с некоторым шагом $h = 0.0025$ и значения функции в точке точного решения, так как положение точного решения задачи есть самая низкая её точка по оси ординат. Таким образом, значение функции в некоторой точке поднималось над осью абсцисс и становилось положительным. А в каждом новом испытании, соответственно, значения минимизируемой на отрезке функции $\varphi(x) \to \min, x \in D = [a, b]$, вычислялись таким образом:

$$\varphi(x) = Hill(x) - Hill(x^*), \text{ где } \varphi(x^*) = 0$$

– для семейства функций Хилла и Гибсона, где $Hill(x)$ – значение функции на новом испытании x, а $Hill(x^*)$ – значение функции при подстановке её точного решения;

$$\varphi(x) = Shekel(x) - Shekel(x^*), \text{ где } \varphi(x^*) = 0$$

– для семейства функций Шекеля, где $Shekel(x)$ – значение функции на новом испытании x, а $Shekel(x^*)$ – значение функции при подстановке её точного решения. Такой методикой мы подняли функции вверх и, следовательно, глобальные минимумы функций оказались в значениях $\varphi(x^*) = 0$, что соответствует нулям функций, которые ищет АПН.

В ходе работы алгоритмов, мы можем применять три разных критерия остановки, согласно которым испытания прекращаются:

1) По точности:

$$x_t - x_{t-1} \leqslant \varepsilon,$$

где (x_{t-1}, x_t) – интервал, которому соответствует максимальная характеристика(содержит точку нового испытания) (см. вычислительные схемы алгоритмов п. 3.1, 4.1, 5.1).

2) По попаданию в окрестность известного значения:

$$|x_t - x^*| \leqslant \varepsilon,$$

где x_t – правая граница найденного интервала, содержащего точку нового испытания (см. вычислительные схемы алгоритмов п. 3.1, 4.1, 5.1), а x^* – точное решение данной рассматриваемой функции семейства.

3) По значению функции:

$$\varphi(x_t) \leqslant \varepsilon,$$

где x_t – правая граница найденного интервала, содержащего точку нового испытания.

С целью рассмотрения работы критериев, применим их на одной задаче из семейства функций Хилла и Гибсона. Возьмем функцию под номером 725 и изобразим её графически, а также отметим на графике результат работы алгоритмов(Рисунок 2: первый ряд штрихов – точки испытаний, которые проводил АГП, второй ряд штрихов – АПН, третий ряд – АДОК) с критерием остановки по попаданию в окрестность известного значения (пункт 2)). Точное решение данной задачи нам известно: $x = 0.215154$

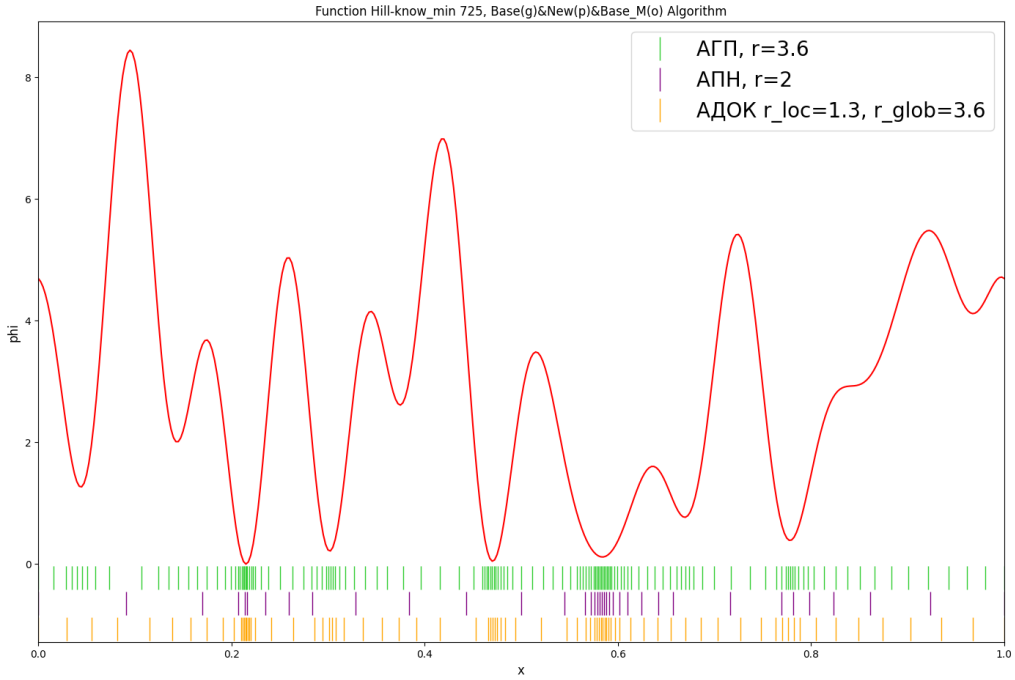


Рисунок 2 – Результат работы АГП, АДОК, АПН с критерием остановки по попаданию в окрестность известного решения

Для наглядности, занесем в таблицу (Таблица 1) количество итераций, потребовавшихся алгоритмам, чтобы решить данную задачу с применением всех трех критериев остановки(1), 2), 3)).

Таблица 1 – Результаты работы алгоритмов с различными критериями остановки

	АГП	**АДОК**	**АПН**
По точности	167	101	39
По попаданию в окр. изв. решения	138	79	37
По значению функции	167	79	40

Конкретную задачу можно решать с использованием подходящего или более оптимального условия остановки. Выбор критерия будет зависеть от рассматриваемой задачи и используемого метода для поиска её глобального минимума.

Если говорить о всей серии тестовых задач, далее рассмотрим работу методов с критерием остановки по известному значению(п. 2)) и по значению функции(п. 3)). Вообще говоря, так как в числе рассматриваемых алгоритмов есть алгоритм поиска нуля, то критерий с выходом по значению функции больше соответствует решаемой задаче.

Итак, необходимые данные для построения операционных характеристик высчитываются в программе [4] и записываются в файл для дальнейшей работы. Перед запуском программы зададим необходимые параметры:

- Параметр метода r:

  АГП $r = 3.6$, АДОК $r_{loc} = 1.3$,$r_{glob} = 3.6$, АПН $r = 2$;

Семейство функций Хилла и Гибсона. Выполним построение графиков и исследуем поведение каждого алгоритма с критерием остановки по попаданию в окрестность известного значения. Точность в данном семействе установлена $\varepsilon = 0.001$.

На представленном графике(Рисунок 3) зеленая линия отвечает за операционные характеристики АГП, желтая – АДОК, сиреневая – АПН. Параметры методов представлены в правом нижнем углу графика.

Здесь мы видим, что зависимость, отвечающая за базовый алгоритм глобального поиска(АГП), плавно растет и достигает 100% при $K = 160$. Это говорит о том, что метод находит глобальный минимум у всех функций семейства за 160 итераций. У 50% функций минимум находится за 71 итерацию.

Далее взглянем на функцию, отвечающую за операционные характеристики алгоритма с двойной оценкой константы Липшица(АДОК). Можно заметить, что примерно у 5% функций глобальный минимум найден за 20 итераций. Далее метод работает быстрее: за

[4]код программы с реализацией функций приведен в приложении.

37 итераций метод нашел решения уже у 50% функций. У всех функций семейства решения были найдены за 100 итераций.

Если посмотреть на зависимость, отвечающую за алгоритм поиска нуля(АПН), то можно отметить очень быстрый рост: уже у 50% функций метод находил решения за 24 итерации. У 98% функций глобальный минимум был найден примерно за 60 итераций. Однако 100% метод достиг только за 110 итераций.

Сравнивая операционные характеристики всех алгоритмов, можно отследить, что, к примеру, у 95% функций: АГП нашел решение примерно за 110 итераций, АДОК примерно за 61 итерацию, АПН примерно за 47 итераций. У 100%: АГП – 160 итераций, АДОК – 100 итераций, АПН – 110 итераций.

Следовательно, хотя АДОК по сравнению с другими двумя алгоритмами находит глобальный минимум у 100% функций за меньшее число итераций(100), все-таки лучше себя показал алгоритм поиска нуля. Его показатели говорят о высокой скорости работы метода, у большей части функций он находит решения гораздо быстрее, чем АГП или АДОК.

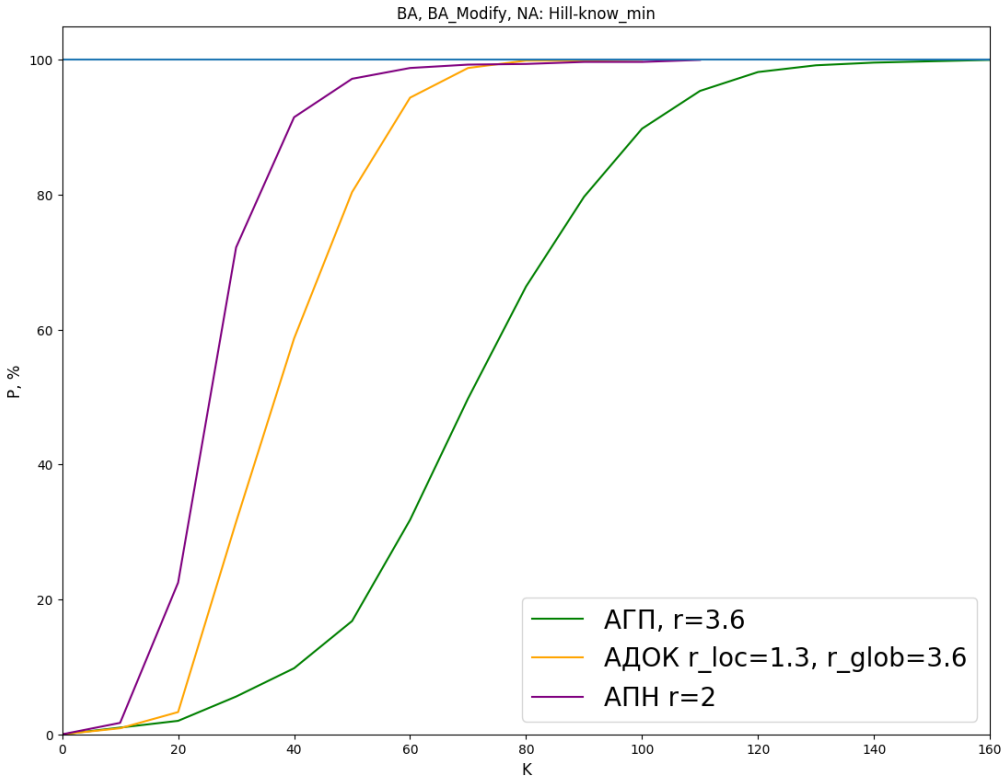


Рисунок 3 – Семейство функций Хилла и Гибсона

Применим критерий остановки по значению функции и также выполним построение графиков операционных характеристик (Рисунок 4). Результаты показали, что алгоритм глобального поиска достигает 100% решенных задач за 210 итераций, алгоритм с двойной оценкой константы Липшица – за 120 итераций, а алгоритм поиска нуля затратил 110. У 50% функций алгоритмы нашли глобальные минимумы за 88(АГП), 56(АДОК), 26(АПН) итераций.

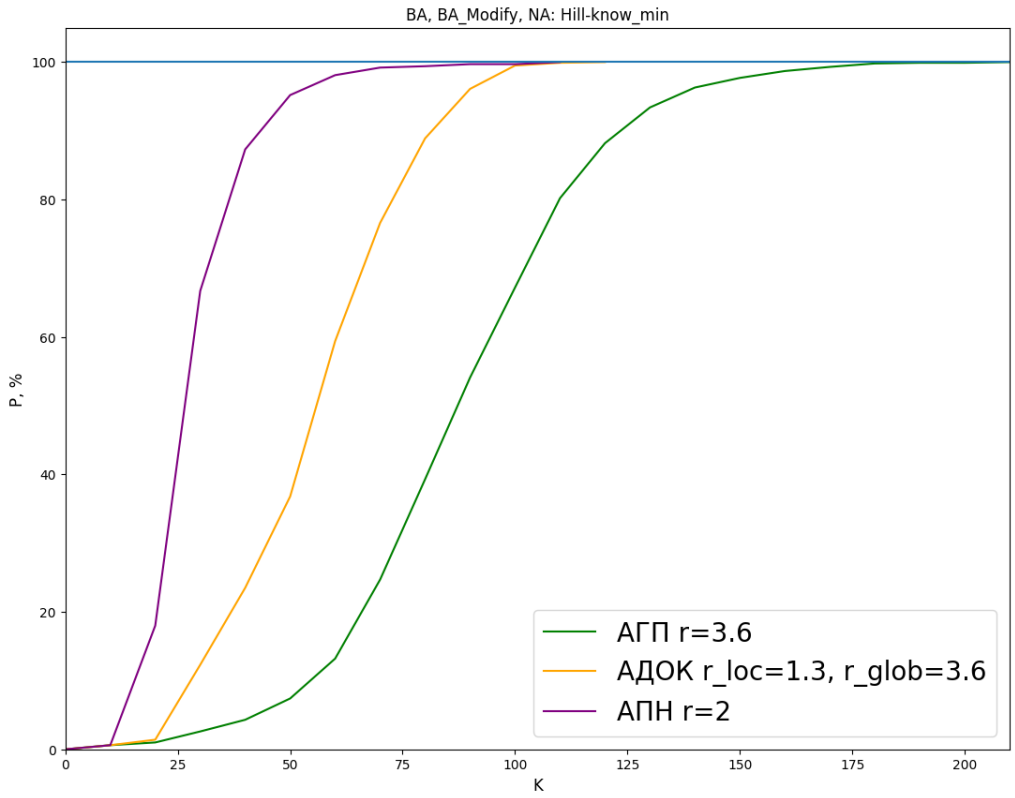


Рисунок 4 – Семейство функций Хилла и Гибсона

Семейство функций Шекеля. Далее посмотрим на график операционных характеристик алгоритмов для семейства функций Шекеля(Рисунок 5) с критерием остановки по попаданию в окрестность известного значения. Отметим, что точность для данного семейства была установлена $\varepsilon = 0.01$. График, отвечающий за АГП, имеет постепенный рост и достигает 100% за 54 итерации. Решения у 50% функций алгоритм находит за 24 итерации.

Зависимость на графике, отвечающая за операционные характеристики алгоритма с двойной оценкой константы Липшица(АДОК), отражает довольно высокую скорость работы метода. У 50% функций глобальный минимум был найден за 17 итераций, а у 100% функций за 49 итераций.

Теперь обратим внимание на функцию, отвечающую за АПН. Её стремительный рост также говорит о высокой скорости работы алгоритма. Отметим, что у 50% функций решения были найдены уже за 15 итераций, а у 100% за 69 итераций. Причем отметим, что у 98% функций решения были найдены всего лишь за 30 итераций. Обратим внимание на это наблюдение чуть позже.

Проведя такое же сравнение, как в семействе функций Хилла и Гибсона, отметим, что у 95% функций семейства Шекеля решения были найдены за: АГП – около 38 итераций,

АДОК – примерно 31 итерация, АПН – 25 итераций. У 100%: АГП – 54 итерации, АДОК – 49 итераций, АПН – 69 итераций.

Следовательно, можно сделать похожий вывод: в данном семействе функций АДОК быстрее справляется в нахождении глобального минимума у всего набора (100%) функций(49 итераций). Однако отметим, что тем не менее на основной массе функций лучше себя показал алгоритм поиска нуля. Его показатели говорят о высокой скорости работы метода, у большей части функций он находит решения гораздо быстрее, чем АГП или АДОК.

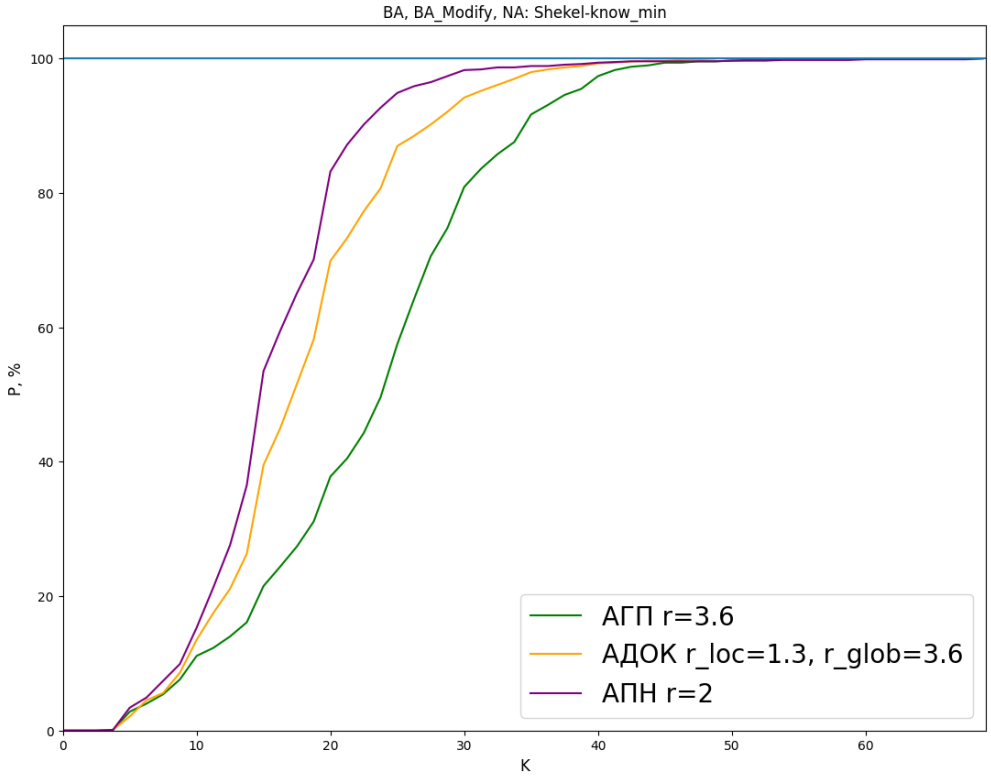


Рисунок 5 – Семейство функций Шекеля

Теперь воспользуемся остановкой методов по значению функции и построим график операционных характеристик(Рисунок 6). АГП на решение каждой из 1000 задач потребовалось 77 итераций, АДОК затратил на это 55 итераций, АПН – 54. У половины от всей массы тестовых задач глобальные минимумы были достигнуты за 37(АГП), 23(АДОК), 17(АПН) итераций.

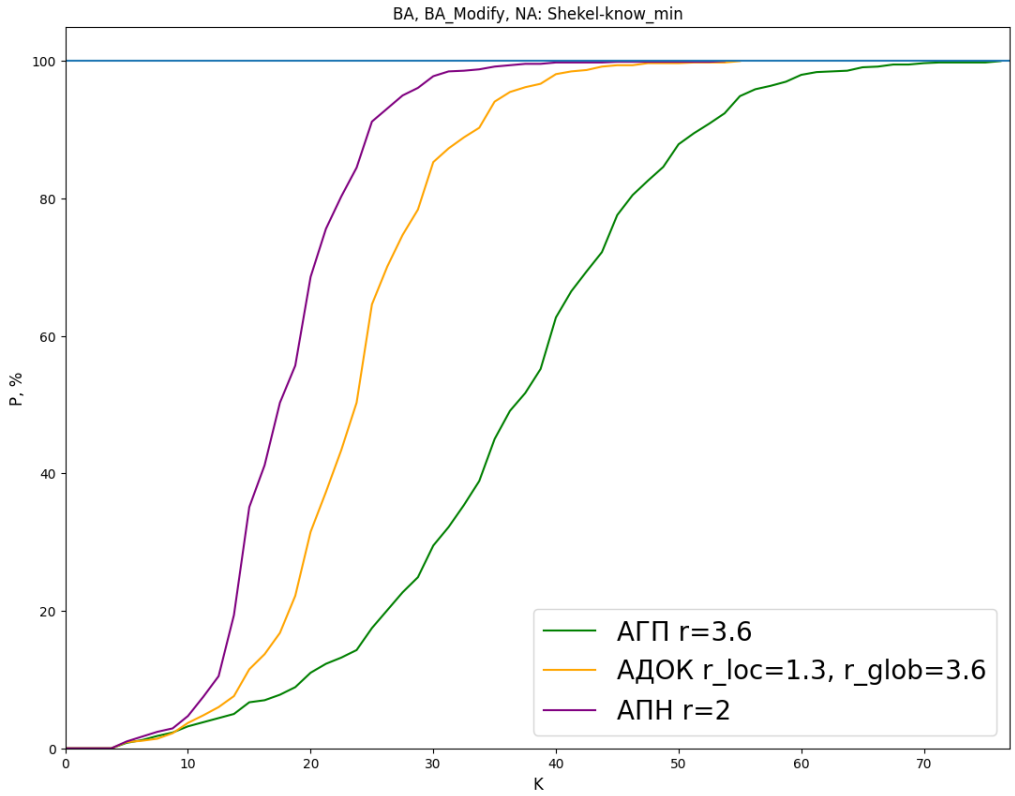


Рисунок 6 – Семейство функций Шекеля

Обратим внимание, что все три рассматриваемых алгоритма справляются быстрее в поиске решений у семейства функций Шекеля, нежели у Хилла и Гибсона.

Рассмотрим табличные данные(Таблица 2), которые подтверждают данное наблюдение, а также предоставляют больше информации для сравнения работы методов. Числа без скобок – показатели, полученые с использованием критерия остановки по попаданию в окрестность известного значения, а числа в скобках – показатели, полученые с использованием критерия остановки по значению функции.

Здесь под «Средним числом итераций» понимается сумма всех испытаний, потребовавшихся, чтобы найти решение у каждой из 1000 функций семейства, деленная на количество всех функций соответственно.

Под «Медианой» понимается число итераций, при котором метод находит решения ровно у половины функций из всего набора.

Таблица 2 – Количественные характеристики методов

	Семейство функций Хилла	**Семейство функций Шекеля**
Среднее число итераций АГП	72 (89)	23 (36)
Медиана АГП	71 (88)	24 (37)
Среднее число итераций АДОК	39 (56)	18 (24)
Медиана АДОК	37 (56)	17 (23)
Среднее число итераций АПН	28 (29)	16 (18)
Медиана АПН	24 (26)	15 (17)

Табличные данные подтверждают, что при рассмотрении двух семейств функций, наилучшие показатели дает алгоритм поиска нуля. Алгоритм с двойной оценкой константы Липшица занял второе место по эффективности и скорости – его показатели ненамного хуже. Базовый алгоритм глобального поиска в сравнении с двумя другими занял третье место.

Выше было обращено внимание на одно наблюдение: на графиках операционных характеристик можно было заметить, что все методы примерно у 98% функций находили решения довольно быстро, а дальше показатели какое-то время переставали увеличиваться и только позже достигали 100%. Такое поведение обусловлено тем, что некоторые функции семейств по определенным причинам являются, так сказать, «проблемными» для алгоритмов. Рассмотрим данный аспект подробнее.

7 «Проблемные» функции для алгоритмов

В данном разделе приведем результаты решения задач, которые оказались сложными для рассматриваемых алгоритмов. Под «проблемной» или, другими словами, сложной функцией, подразумевается такая функция из рассматриваемого семейства, на поиск решения которой методу потребовалось больше всего итераций.

Отметим, что параметр метода r был задан: АГП $r = 3.6$, АДОК $r_{loc} = 1.3$,$r_{glob} = 3.6$, АПН $r = 2$ соответственно.

Количество функций в обоих семействах – 1000.

Будем проводить эксперименты с условием остановки алгоритмов по попаданию в окрестность известного значения (п. 2)) и с условием остановки по значению функции (п. 3)).

Семейство функций Хилла и Гибсона.

Те функции данного семейства, которые будут рассмотрены ниже, представляют сложность для методов по причине того, что имеют большое количество похожих друг на друга минимумов. Локальные минимумы во многих примерах расположены примерно на одной высоте с глобальным, что затрудняет поиск решения и требует большего числа итераций.

Алгоритм глобального поиска. В данном семействе точность была задана $\varepsilon = 0.001$. Рассмотрим самую сложную функцию для базового алгоритма глобального поиска. Такой для АГП, с критерием остановки по попаданию в окрестность известного значения, оказалась функция под номером 77 в семействе.

Результат работы программы [5] по поиску глобального минимума задачи с использованием АГП:

Текущий результат:

$$z = 0.00198922$$
$$x = 0.229004$$
$$k = 159$$

Лучший результат:

$$z = 0.000568465$$
$$x = 0.229708$$
$$k = 121$$

Точное решение: $x = 0.229463$,

z – значение функции, x – аргумент функции, k – номер испытания.

По программным выкладкам можно увидеть, что метод затратил 159 итераций, прежде чем найти решение у функции. Также программой были получены результаты работы двух других алгоритмов. Алгоритму с двойной оценкой константы Липшица потребовалось 50 итераций, чтобы решить задачу. Алгоритму поиска нуля – 41 итерация.

[5] код программы с реализацией функций приведен в приложении.

На представленном графике функции под номером 77(Рисунок 7): первый ряд штрихов – испытания, которые проводил АГП, второй ряд – АПН, третий – АДОК.

На графике отчетливо видно, что АГП действительно провел достаточно много испытаний, причем зоны сгущения – области, подозрительные на наличие глобального минимума.

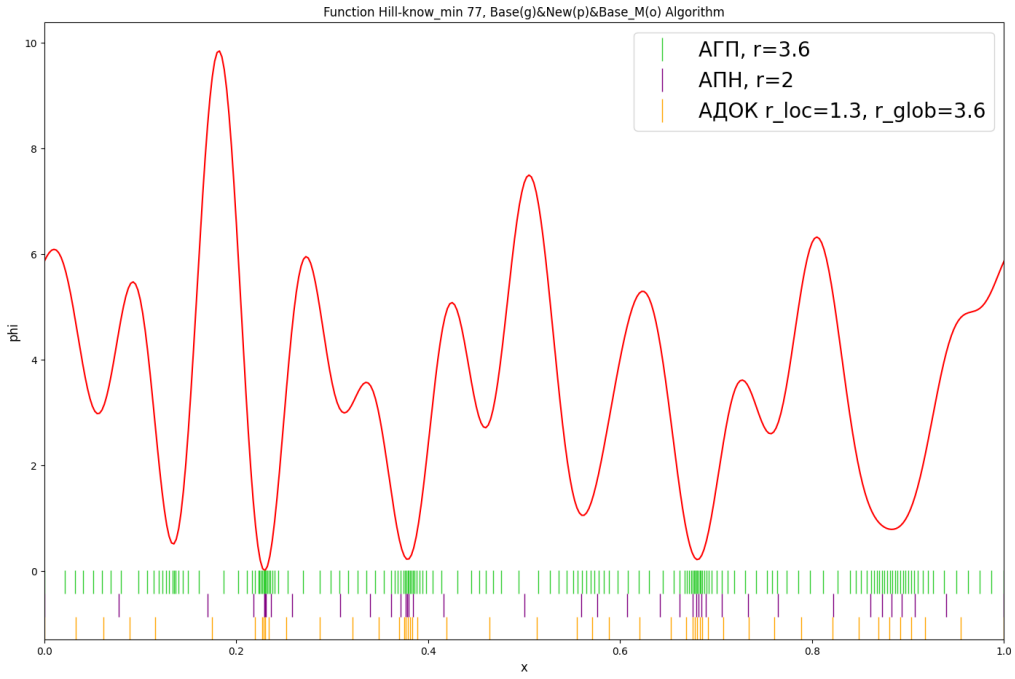


Рисунок 7 – Результат работы АГП, АДОК, АПН

Для АГП, с критерием остановки по значению функции, сложной оказалась задача под номером 301 в семействе.

Результат работы программы по поиску глобального минимума задачи с использованием АГП:

Текущий результат:

$$z = 0.000695555$$
$$x = 0.668983$$
$$k = 205$$

Лучший результат:

$$z = 1.48378e-06$$
$$x = 0.669285$$
$$k = 172$$

Точное решение: $x = 0.669272$,

z – значение функции, x – аргумент функции, k – номер испытания.

Итак, рассматриваемый метод затратил 205 итераций на поиск решения, а АДОК и АПН – 93 и 109. На представленном рисунке(Рисунок 8) также видны зоны сгущения в местах, подозрительных на глобальный минимум функции.

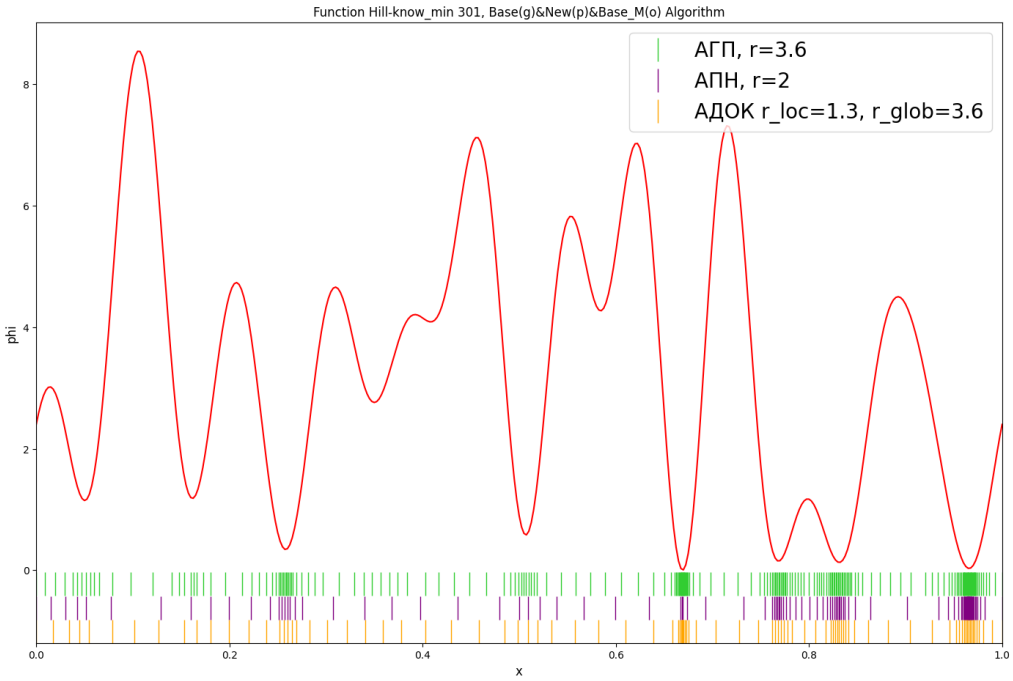


Рисунок 8 – Результат работы АГП, АДОК, АПН

Алгоритм с двойной оценкой константы Липшица. Далее рассмотрим самую сложную функцию для алгоритма с двойной оценкой константы Липшица: Такой для АДОК, с критерием остановки по попаданию в окрестность известного значения, оказалась функция под номером 301 в семействе.

Результат работы программы по поиску глобального минимума задачи с использованием АДОК:

Текущий результат:

$$z = 0.00614639$$
$$x = 0.668414$$
$$k = 93$$

Лучший результат:

$$z = 0.000407072$$
$$x = 0.669051$$
$$k = 68$$

Точное решение: $x = 0.669272$,

z – значение функции, x – аргумент функции, k – номер испытания.

По результатам работы программы видно, что метод затратил 93 итерации, прежде чем найти глобальный минимум данной функции. В свою очередь, базовому алгоритму глобального поиска потребовалось 47 итераций, чтобы решить задачу, а алгоритму поиска нуля – 109 итераций. Отметим, что для АПН функция тоже оказалась довольно сложной, исходя из количества затраченных итераций.

На представленном графике изображена функция под номером 301 и результаты работы трех рассматриваемых алгоритмов(Рисунок 9): первый ряд штрихов – испытания, которые проводил АГП, второй ряд – АПН, третий – АДОК.. Видим, что АДОК проводил

больше испытаний в местах, подозрительных на глобальный минимум данной функции, этому свидетельствуют зоны сгущения на графике.

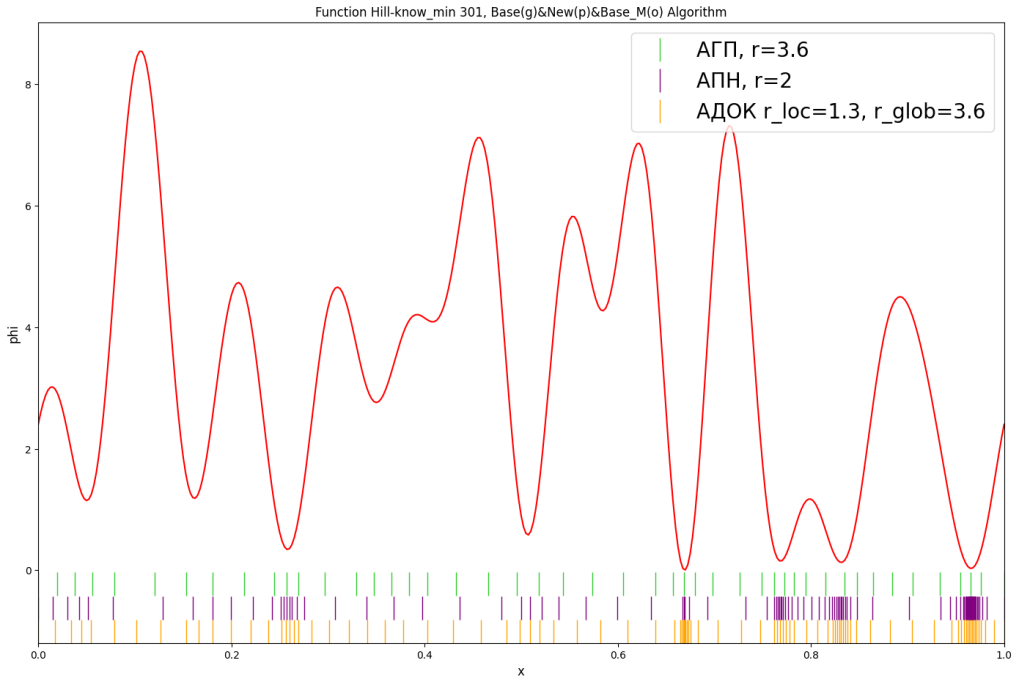


Рисунок 9 – Результат работы АГП, АДОК, АПН

Для АДОК, с критерием остановки по значению функции, сложной оказалась задача под номером 649 в семействе.

Результат работы программы по поиску глобального минимума задачи с использованием АДОК:

Текущий результат:

$$z = 0.000352015$$
$$x = 0.286594$$
$$k = 174$$

Лучший результат:

$$z = 8.21642e - 06$$
$$x = 0.286804$$
$$k = 151$$

Точное решение: $x = 0.286776$,

z – значение функции, x – аргумент функции, k – номер испытания.

По программным выкладкам видно, что метод затратил 113 итераций на поиск решения. АГП и АПН потребовалось 174 и 38 итераций. На графике (Рисунок 10) представлен вид функции и результат работы алгоритмов.

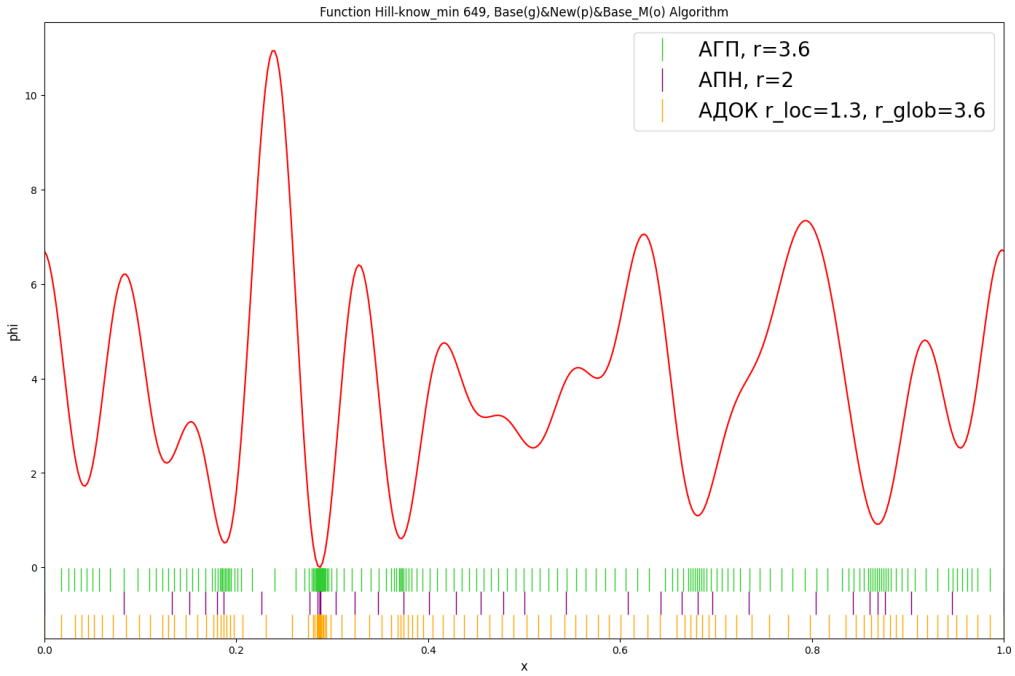


Рисунок 10 – Результат работы АГП, АДОК, АПН

Алгоритм поиска нуля. Теперь рассмотрим самую сложную функцию для алгоритма поиска нуля. Такой для АПН, с критерием остановки по попаданию в окрестность известного значения, оказалась функция под номером 385 в семействе.

Результат работы программы по поиску глобального минимума задачи с использованием АПН:

Текущий результат:

$$z = 2.44963e-05$$
$$x = 0.963301$$
$$k = 110$$

Лучший результат:

$$z = 2.44963e-05$$
$$x = 0.963301$$
$$k = 110$$

Точное решение: $x = 0.963356$,

z – значение функции, x – аргумент функции, k – номер испытания.

Исходя из программных выкладок, метод затратил 110 итераций, прежде чем найти решение задачи. Базовому алгоритму глобального поиска потребовалось на это 145 итераций, а алгоритму с двойной оценкой константы Липшица – 75.

На представленном графике изображена функция под номером 385 и результаты работы трех рассматриваемых алгоритмов(Рисунок 11): первый ряд штрихов – испытания, которые проводил АГП, второй ряд – АПН, третий – АДОК..

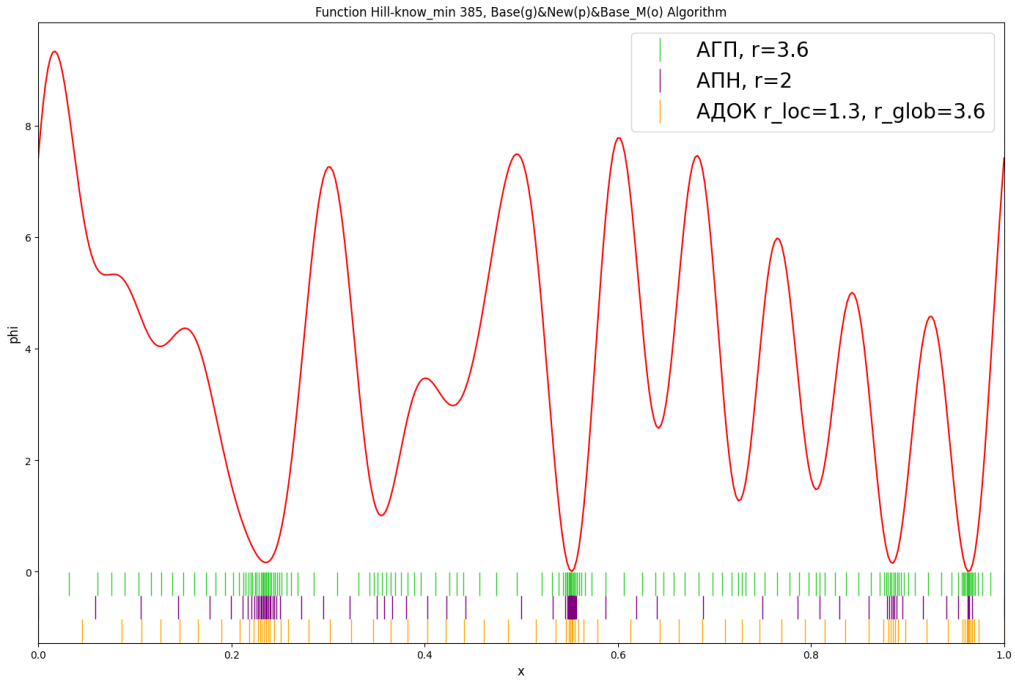


Рисунок 11 – Результат работы АГП, АДОК, АПН

Для АПН с критерием остановки по значению функции сложной оказалась все та же задача под номером 385 в семействе(Рисунок 11). Число итераций, потребовавшихся каждому методу для решения задачи, не изменилось: АГП – 145 итераций, АДОК – 75, АПН – 110.

Семейство функций Шекеля.

Следует отметить, что сложность представленных ниже функций из семейства Шекеля заключается в том, что они имеют слишком «узкий» глобальный минимум. На начальном этапе методы расставят точки на склонах данного минимума и подумают, что в этом месте большие значения. Из-за сильного наклона функции, это место даст большую константу Липшица. Методы продолжат ставить точки испытаний в других местах подозрительных на глобальный минимум. Это будет происходить до момента, пока интервалы и характеристика не станут достаточно маленькими и не посчитается новая константа там, где поблизости находится решение задачи. Такая сложность требует большего числа итераций, чтобы найти ответ.

Алгоритм глобального поиска. При заданной точности $\varepsilon = 0.01$ для данного семейства функций, функции под номером 484 и 570 оказались наиболее сложными для оптимизации с использованием базового алгоритма глобального поиска, требуя максимального количества итераций. Отметим, что выход производился по попаданию в окрестность известного значения. Рассмотрим функцию под номером 484 в семействе.

Результат работы программы по поиску глобального минимума задачи с использованием АГП:

Текущий результат:

$$z = 0.072641$$
$$x = 6.14463$$
$$k = 53$$

Лучший результат:

$$z = 4.38824e - 07$$
$$x = 6.19064$$
$$k = 33$$

Точное решение: $x = 6.191$,

z – значение функции, x – аргумент функции, k – номер испытания.

Отсюда можем отметить, что АГП потребовалось 53 итерации, чтобы найти глобальный минимум у данной задачи. АДОК затратил 46 итераций, а АПН – 28.

На представленном графике изображена функция под номером 484 и результаты работы трех рассматриваемых алгоритмов(Рисунок 12): первый ряд штрихов – испытания, которые проводил АГП, второй ряд – АПН, третий – АДОК.

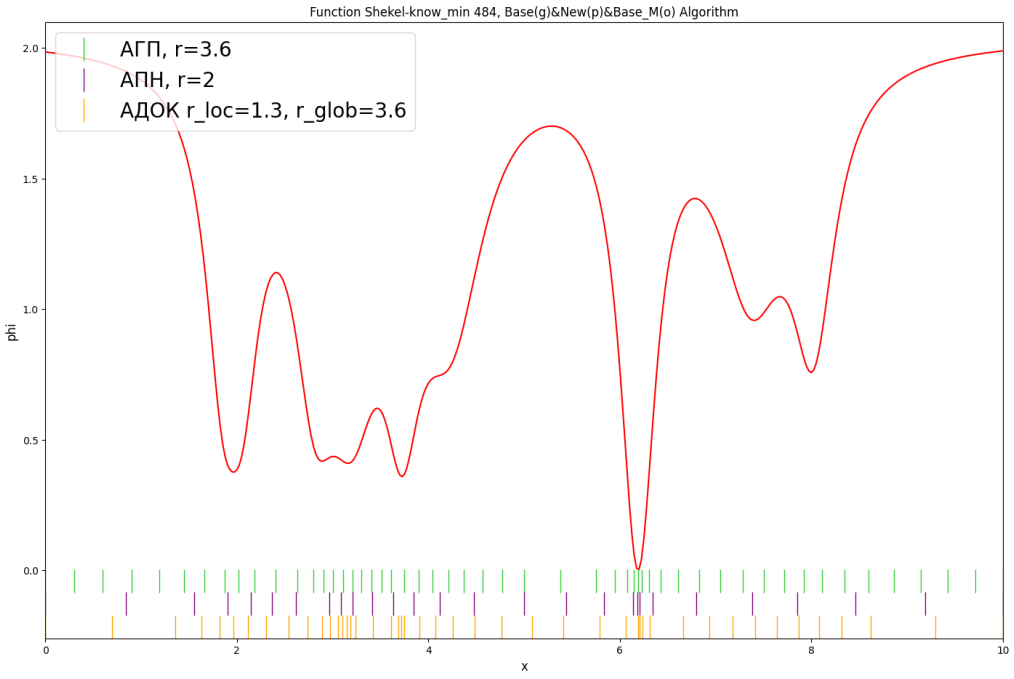


Рисунок 12 – Результат работы АГП, АДОК, АПН

Перейдем к рассмотрению функции под номером 570 в семействе.

Результат работы программы по поиску глобального минимума задачи с использованием АГП:

Текущий результат:

$$z = 0.213806$$
$$x = 1.2488$$
$$k = 53$$

Лучший результат:

$$z = 0.00205659$$
$$x = 1.34159$$
$$k = 49$$

На поиск решения АГП затратил также 53 итерации, АДОК – 48 итераций, а АПН – 53.

На представленном графике изображена функция под номером 570 и результаты работы трех рассматриваемых алгоритмов(Рисунок 13): первый ряд штрихов – испытания, которые проводил АГП, второй ряд – АПН, третий – АДОК..

В соответствии с графиком, есть несколько зон сгущения штрихов и часть из них находятся не в окрестности глобального минимума. Это говорит о том, что алгоритм рассматривал еще и другую область, ошибочно полагая, что там находится решение.

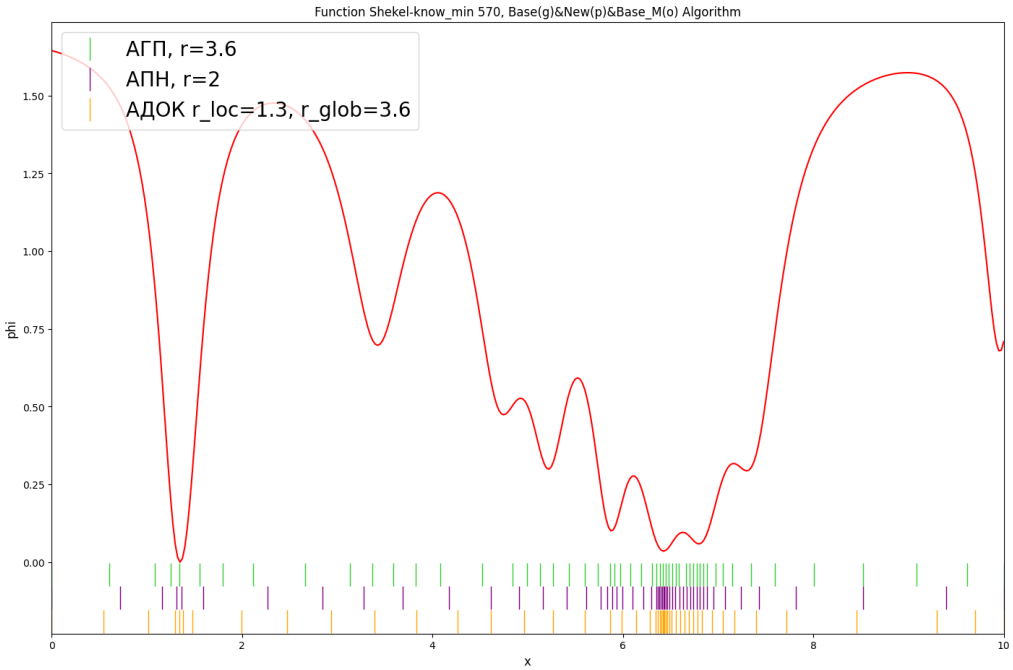


Рисунок 13 – Результат работы АГП, АДОК, АПН

Для АГП с критерием остановки по значению функции сложными оказались задачи под номером $277,832$ в семействе.

Результат работы программы по поиску глобального минимума задачи под номером 277 с использованием АГП:

Текущий результат:

$$z = 0.00436322$$
$$x = 9.64696$$
$$k = 76$$

Лучший результат:

$$z = 2.8367\text{e} - 05$$
$$x = 9.65708$$
$$k = 62$$

Точное решение: $x = 9.658$,

z – значение функции, x – аргумент функции, k – номер испытания.

По программным выкладкам можно увидеть, что метод затратил 76 итераций на поиск решения. АДОК и АПН потребовалось 44 и 34 итерации. На представленном графике(Рисунок 14) изображен вид функции под номером 277.

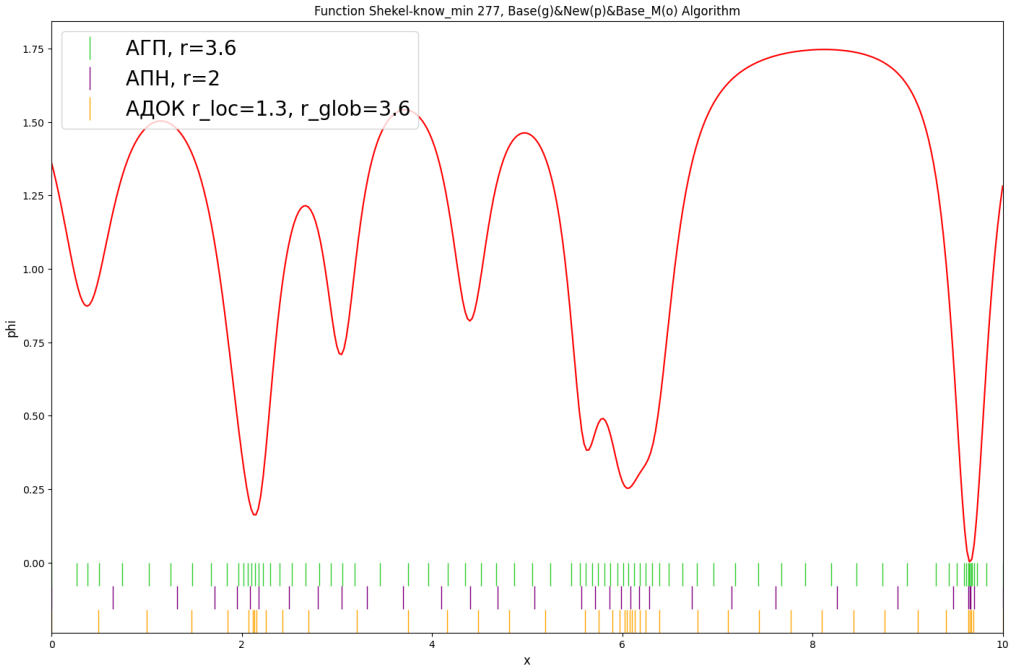


Рисунок 14 – Результат работы АГП, АДОК, АПН

Результат работы программы по поиску глобального минимума задачи под номером 832 с использованием АГП:

Текущий результат:

$$z = 0.00415859$$
$$x = 9.5514$$
$$k = 76$$

Лучший результат:

$$z = 9.11828e - 05$$
$$x = 9.56073$$
$$k = 60$$

Точное решение: $x = 9.652$,

z – значение функции, x – аргумент функции, k – номер испытания.

АГП затратил также 76 итераций на поиск решения. АДОК и АПН потребовалось 45 и 30 итерации (Рисунок 15).

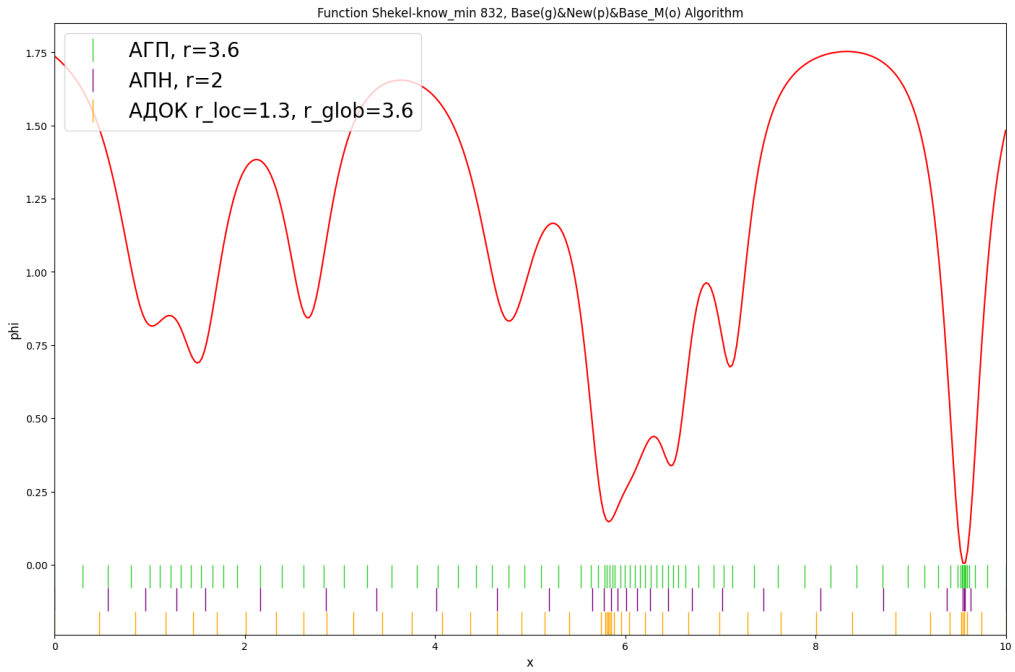


Рисунок 15 – Результат работы АГП, АДОК, АПН

Алгоритм с двойной оценкой константы Липшица. Наиболее сложной функцией для АДОК с критерием остановки по попаданию в окрестность известного значения оказалась задача под номером 570 в семействе, она же является одной из сложных для АГП (см. Рисунок 13).

Результат работы программы по поиску глобального минимума задачи с использованием АДОК:

Текущий результат:

$$z = 0.00187102$$
$$x = 1.34204$$
$$k = 48$$

Лучший результат:

$$z = 0.00187102$$
$$x = 1.34204$$
$$k = 48$$

Точное решение: $x = 1.351$,

z – значение функции, x – аргумент функции, k – номер испытания.

На поиск решения АДОК затратил 48 итераций, АГП – 53 итерации, а АПН – 53.

Для АДОК с критерием остановки по значению функции проблематичной оказалась функция под номером 374 в семействе, а также все та же задача под номером 570, однако с другими результатами работы.

Результат работы программы по поиску глобального минимума задачи под номером 374 с использованием АДОК:

Текущий результат:

$$z = 0.00358314$$
$$x = 8.92533$$
$$k = 54$$

Лучший результат:

$$z = 9.17415\text{e} - 06$$
$$x = 8.93765$$
$$k = 50$$

Точное решение: $x = 8.937$,

z – значение функции, x – аргумент функции, k – номер испытания.

По программным выкладкам можно увидеть, что метод затратил 54 итерации на поиск решения. АГП и АПН, в свою очередь, потребовалось 65 и 34 итерации. На представленном графике изображена функция под номером 374 и результаты работы трех рассматриваемых алгоритмов(Рисунок 16): первый ряд штрихов – испытания, которые проводил АГП, второй ряд – АПН, третий – АДОК..

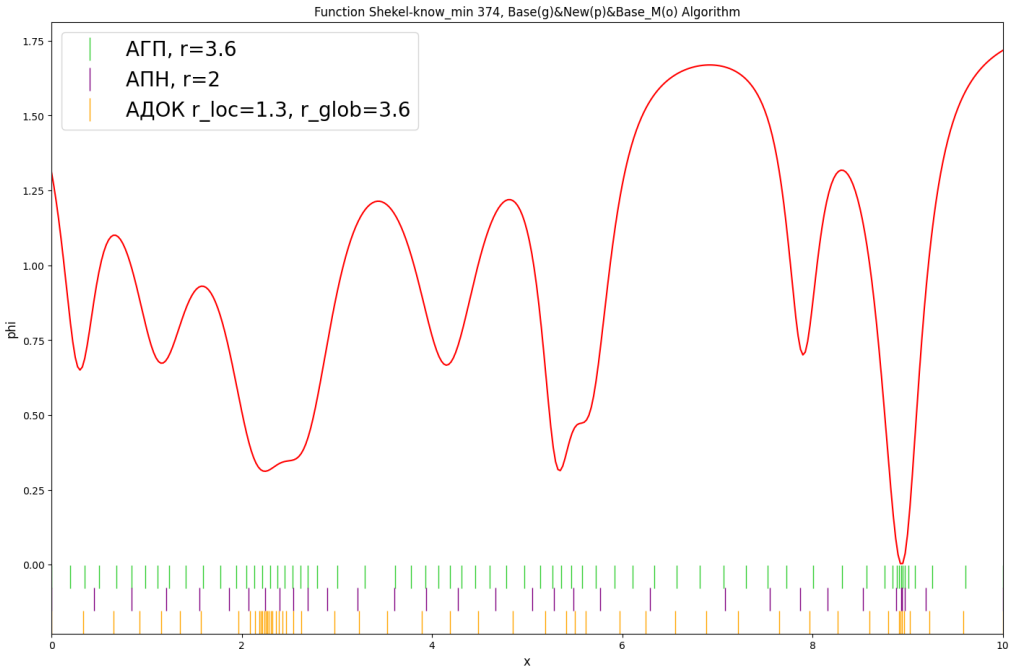


Рисунок 16 – Результат работы АГП, АДОК, АПН

Результат работы программы по поиску глобального минимума задачи под номером 570 с использованием АДОК:

Текущий результат:

$$z = 0.0171741$$
$$x = 1.32312$$
$$k = 54$$

Лучший результат:

$$z = 0.00187102$$
$$x = 1.34204$$
$$k = 48$$

Точное решение: $x = 1.351$,

z – значение функции, x – аргумент функции, k – номер испытания.

По программным выкладкам можно увидеть, что метод затратил 54 итерации на поиск решения. АГП и АПН, в свою очередь, потребовалось 65 и 34 итерации. На представленном рисунке(Рисунок 17) видны зоны сгущения в местах, подозрительных на глобальный минимум задачи под номером 570.

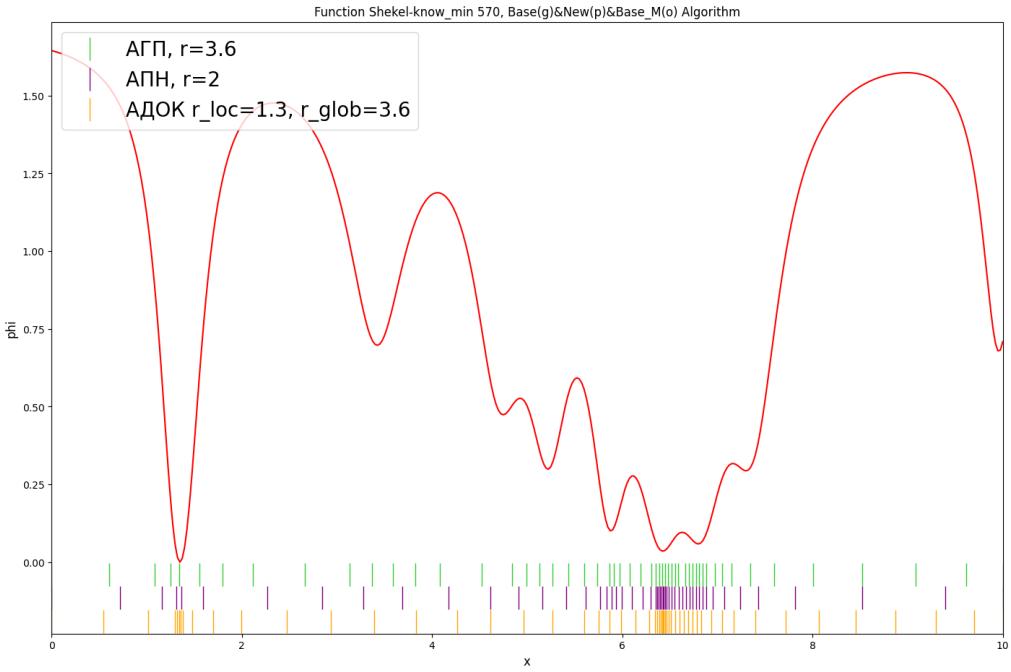


Рисунок 17 – Результат работы АГП, АДОК, АПН

Алгоритм поиска нуля. Далее проанализируем функцию, которая представляла наибольшие трудности для алгоритма поиска нуля в процессе нахождения глобального экстремума. С использованием критерия остановки по попаданию в окрестность известного значения, таковой оказалась функция под номером 934 в семействе.

Результат работы программы по поиску глобального минимума задачи с использованием АПН:

Текущий результат:

$$z = 0.000324518$$
$$x = 9.51962$$
$$k = 68$$

Лучший результат:

$$z = 0.000324518$$
$$x = 9.51962$$
$$k = 68$$

Точное решение: $x = 9.524$,

z – значение функции, x – аргумент функции, k – номер испытания.

По результатам работы программы видно, что метод затратил 68 итераций, прежде чем найти глобальный минимум данной функции. В свою очередь, алгоритму с двойной оценкой константы Липшица потребовалось 25 итераций, чтобы решить задачу. Базовый алгоритм глобального поиска затратил 39 итераций, найдя очень похожий на глобальный минимум – локальный в точке $x = 6.51427$. В ходе экспериментов выяснилось, что при уменьшении точности до $\varepsilon = 0.001$ метод справляется с задачей и находит глобальный минимум как в точном решении данной тестовой задачи.

На представленном графике изображена функция под номером 934 и результаты работы трех рассматриваемых алгоритмов(Рисунок 18): первый ряд штрихов – испытания,

которые проводил АГП, второй ряд – АПН, третий – АДОК. На графике видно, что АПН провел больше всего испытаний в двух местах, это можно пронаблюдать по зонам сгущения штрихов.

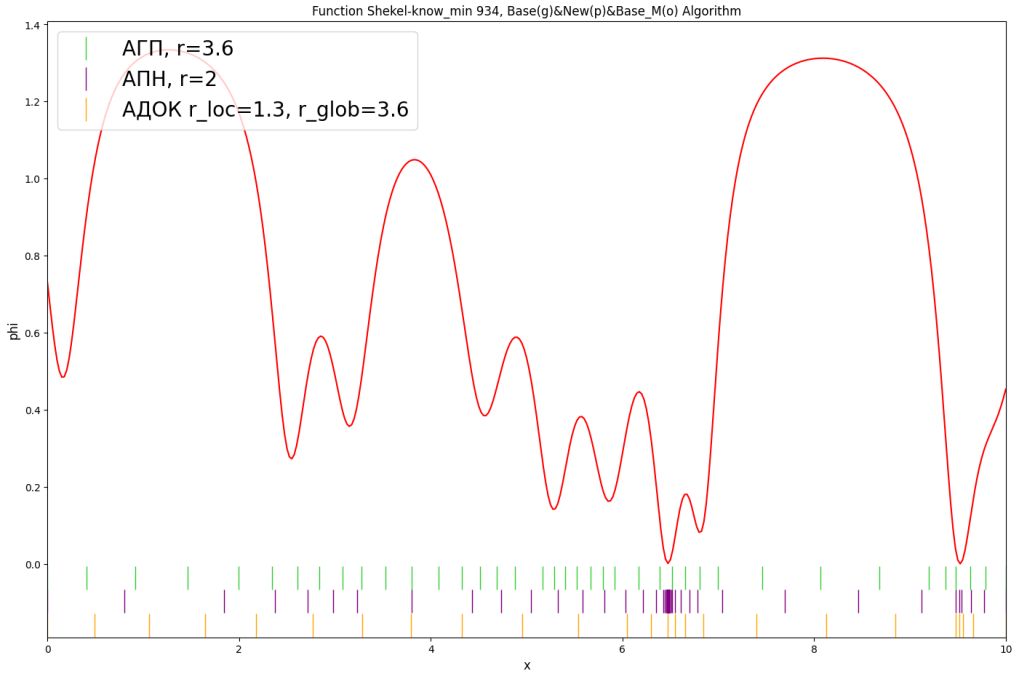


Рисунок 18 – Результат работы АГП, АДОК, АПН

У АПН с критерием остановки по значению функции потребовалось больше всего итераций на задачу под номером 570 в семействе. Эта же функция была выявлена как самая сложная для алгоритма с двойной оценкой константы Липшица.

Результат работы программы по поиску глобального минимума задачи с использованием АПН:

Текущий результат:

$$z = 0.0288101$$
$$x = 1.31477$$
$$k = 53$$

Лучший результат:

$$z = 0.00437265$$
$$x = 1.36585$$
$$k = 52$$

Точное решение: $x = 1.351$,

z – значение функции, x – аргумент функции, k – номер испытания.

Видно, что данный метод решил задачу за 53 итерации. Что касается АГП и АДОК – методы управились с данной задачей за 53 и 54 итерации (см. Рисунок 17).

Таким образом, в данном разделе было проведено исследование самых проблемных функций обоих семейств для рассматриваемых алгоритмов и выявлены причины такого поведения.

Заключение

В данной работе было проведено исследование задач оптимизации и методов их решения, реализованы алгоритмы глобальной оптимизации для решения задач многоэкстремальной оптимизации. В число реализованных методов вошли: базовый алгоритм глобального поиска(АГП), алгоритм с двойной оценкой константы Липшица(АДОК), алгоритм поиска нуля(АПН). Были проведены вычислительные эксперименты на двух функциональных семействах: семейство тестовых функций Шекеля и семейство тестовых функций Хилла и Гибсона. Эксперименты позволили выявить эффективность каждого из алгоритмов, оценив их операционные характеристики, а также сравнить методы между собой. На основе вычислительных экспериментов также были исследованы и графически отображены такие задачи из каждого семейства, которые для алгоритмов оказались самыми сложными в смысле поиска у них глобального минимума. Для получения всех необходимых результатов была разработана программа, которая содержит в себе реализацию трех рассматриваемых методов, подсчет данных для качественных и количественных операционных характеристик, а также подсчет данных для возможности графического отображения функций и точек испытаний, которые ставили алгоритмы во время поиска наилучшего решения.

Подводя итоги, можно сказать, что главная цель любого алгоритма глобальной оптимизации – поиск наилучшего минимума при условии всех возможных ограничений. Стоит отметить, что методов решения задач многоэкстремальной оптимизации множество, но каждый из них имеет разный уровень эффективности и применимости к той или иной задаче. На основании всех результатов, полученных в ходе вычислительных экспериментов, можно сделать вывод, что в данной работе самым эффективным алгоритмом по поиску глобального минимума оказался алгоритм поиска нуля. Данный новый метод на этапе исследования находил решения у рассматриваемых задач за меньшее число итераций, чем АГП или АДОК. Операционные характеристики также показали, что метод эффективнее справлялся с поиском глобального минимума у задач двух функциональных семейств.

Проведённое исследование вносит вклад в развитие методов глобальной оптимизации, предлагая сравнительный анализ эффективности двух уже изученных алгоритмов с одним новым. Полученные результаты имеют практическое значение для разработки более эффективных оптимизационных решений в прикладных областях и создания гибридных алгоритмов, сочетающих сильные стороны рассмотренных методов. В будущем, для дальнейшего исследования алгоритма поиска нуля, можно адаптировать метод для работы в пространствах высокой размерности с целью расширить класс тестовых функций, добавив многомерные многоэкстремальные задачи оптимизации и включив функции с различными типами поверхности(разрывные, с оврагами и т.д.). Можно включить исследование сходимости алгоритма поиска нуля для различных классов функций и анализ вычислительной сложности метода в зависимости от размерности задачи. В дальнейшем можно провести прикладные исследования, протестировав алгоритм на реальных задачах из инженерной практики, а также интегрировать метод в системы поддержки принятия решений.

Алгоритм поиска нуля показал себя как перспективное направление исследований в

области глобальной оптимизации. Его дальнейшее развитие и адаптация к многомерным случаям могут привести к созданию нового класса высокоэффективных оптимизационных алгоритмов, востребованных в современных вычислительных и инженерных приложениях, а также в различных областях математики, механики.

Литература

1. *Болховитинов А.С.* Оптимизация трассировки топологий самосборки чипов и разработки методов самосборки "лабораторий на чипе". – Ж. наноиндустрия. 2014, т.7, № 7, с.56-63.
2. *Городецкий С.Ю., Гришагин В.А.* Нелинейное программирование и многоэкстремальная оптимизация – Н.Новгород: Изд-во ННГУ, 2007
3. *Жилинкас А.Г., Шалтянис В.Р.* Поиск оптимума – М.: Наука, 1989
4. *Стронгин Р.Г., Гергель В.П.* АБСОЛЮТ. Программная система для исследований и изучения методов глобальной оптимизации. Учебное пособие. Нижний Новгород: Изд-во Нижегородского университета, 1998. 141 с.
5. *Стронгин Р.Г.* Поиск глобального оптимума – М.: Знание, 1990
6. *Стронгин Р.Г.* Численные методы в многоэкстремальных задачах. – М.: Наука, 1978
7. *Strongin, R.G., Sergeyev, Y.D.*: Global Optimization with Non-convex Constraints. Sequential and parallel Algorithms. Kluwer Academic Publishers, Dordrecht (2000).
8. *Roman Strongin, Konstantin Barkalov, Semen Bevzuk*: Acceleration of Global Search by Implementing Dual Estimates for Lipschitz Constant. Lobachevsky State University of Nizhni Novgorod, Nizhny Novgorod
9. *Jamil, M., Yang, X.-S.*(2013) A Literature Survey of Benchmark Functions For Global Optimization Problems

Приложение

Solve.h

```
struct Trial {
   double x, z;
   int k;
};

class Solver
{
protected:
   double eps = 0.001;
   double r = 3.6;

   double a = 0, b = 1;
   int Kmax = 1000;

   Trial BestTrial;
   Trial NewTrial;
   THillProblem hp;
   THillProblemFamily hpf;
   TShekelProblem shp;
   TShekelProblemFamily shf;

public:
   vector<Trial> trials;

   void Solve(int i);
   void FirstTrial(int i, double* minFunc, double* argmin, double* l, static vector<Trial>&
trials);
   void EstimateConstant(int *K, static vector<Trial>& trials, double *m);
   void FindMaxR_BA(int* K, static vector<Trial>& trials, int* t, double* m);

   void FindMaxR_BA_UPD(int* K, double* minFunc, static vector<Trial>& trials, int* t,
double* m); //Base Algorithm
   void MakeNewTrial_BA(int i, int* K, static vector<Trial>& trials, int* t, double* m);
   void UpdateOptimum(double* minFunc, double* argmin, double* l, static vector<Trial>&
trials);
   void CheckStopCondition(bool *flag, static vector<Trial>& trials, int* t);
   void CheckStopConditionWithKnownOptPoint(int i, bool* flag, static vector<Trial>&
trials, int* t);
   void CheckStopConditionCurrentZ(int i, bool* flag, static vector<Trial>& trials, int*
t);
   void MakeBestTrial(double* minFunc, double* argmin, double* l);

   int GetK();
   int GetKcur();
   int h, hh;
   Trial GetSolution();
   int mProblemIndex;

   double obj_func(double x, int i) {
      //return x * x - cos(18 * x) + sin(50 * x) + 1.878638;

      //return hp.ComputeFunction({ x });
```

```
      double known_min = hpf[i]->GetOptimumValue();
      return hpf[i]->ComputeFunction({ x }) - known_min;

      //return shp.ComputeFunction({ x });

      //double known_min = shf[i]->GetOptimumValue();
      //return shf[i]->ComputeFunction({ x }) - known_min;
   }
};

class SolverNew : public Solver
{
public:
   void Solve(int i);
   void FindMinR(int* K, vector<Trial>& trials, int* t);
   void MakeNewTrial(int i, int* K, vector<Trial>& trials, int* t);
};

class SolverModif : public Solver
{
protected:
   double r_glob = 3.6;
   double r_loc = 1.3;
   double ro = ((1 - 1 / r_glob) / (1 - 1 / r_loc)) * ((1 - 1 / r_glob) / (1 - 1 / r_loc));
public:
   void Solve(int i);
   void FindMaxR_BA_Modif(int* K, double* minFunc, vector<Trial>& trials, int* t, double*
m, double* r_upd);
   void MakeNewTrial(int i, int* K, vector<Trial>& trials, int* t, double* m, double*
r_upd);
};

class File {
public:
   void InputToFile_percent(vector<int>&K, int n, int a);
   void InputToFile_x_solver(vector<Trial>& trials, int n, int a);
};

class Practice {
public:
   void BA(vector<int>& K, int n);
   void BA_Modif(vector<int>& K, int n);
   void NewA(vector<int>& K, int n);
};
```

Solve.cpp

```
void Solver::Solve(int i) {
   int K = 2;
   bool flag = 1;
   trials.resize(Kmax);
   double minFunc = trials[0].z, argmin = trials[0].x, l = trials[0].k;
   FirstTrial(i, &minFunc, &argmin, &l, trials);
   while (flag && (K < Kmax)) {
```

```
      double m;
      int t;
      EstimateConstant(&K, trials, &m);

      FindMaxR_BA_UPD(&K, &minFunc, trials, &t, &m);
      MakeNewTrial_BA(i, &K, trials, &t, &m);

      UpdateOptimum(&minFunc, &argmin, &l, trials);
      //CheckStopCondition(&flag, trials, &t);
      //CheckStopConditionWithKnownOptPoint(i, &flag, trials, &t);
      CheckStopConditionCurrentZ(i, &flag, trials, &t);
   }
   MakeBestTrial(&minFunc, &argmin, &l);
   GetSolution();
}

void SolverModif::Solve(int i) {
   int K = 2;
   bool flag = 1;
   trials.resize(Kmax);
   double minFunc = trials[0].z, argmin = trials[0].x, l = trials[0].k;
   FirstTrial(i, &minFunc, &argmin, &l, trials);
   while (flag && (K < Kmax)) {
      double m;
      double r_upd;
      int t;
      EstimateConstant(&K, trials, &m);

      FindMaxR_BA_Modif(&K, &minFunc, trials, &t, &m,&r_upd);
      MakeNewTrial(i, &K, trials, &t, &m, &r_upd);

      UpdateOptimum(&minFunc, &argmin, &l, trials);
      //CheckStopCondition(&flag, trials, &t);
      //CheckStopConditionWithKnownOptPoint(i, &flag, trials, &t);
      CheckStopConditionCurrentZ(i, &flag, trials, &t);
   }
   MakeBestTrial(&minFunc, &argmin, &l);
   GetSolution();
}

void SolverNew::Solve(int i) {
   int K = 2;
   bool flag = 1;
   trials.resize(Kmax);
   double minFunc = trials[0].z, argmin = trials[0].x, l = trials[0].k;
   FirstTrial(i, &minFunc, &argmin, &l, trials);

   while (flag && (K < Kmax)) {
      int t;

      FindMinR(&K, trials, &t);
      MakeNewTrial(i, &K, trials, &t);

      UpdateOptimum(&minFunc, &argmin, &l, trials);
      //CheckStopCondition(&flag, trials, &t);
      //CheckStopConditionWithKnownOptPoint(i, &flag, trials, &t);
```

```
      CheckStopConditionCurrentZ(i, &flag, trials, &t);
   }
   MakeBestTrial(&minFunc, &argmin, &l);
   GetSolution();
}

Trial Solver:: GetSolution() {
   cout << "\nТекущий результат" << endl;
   cout << "z = " << NewTrial.z << endl;
   cout << "x = " << NewTrial.x << endl;
   cout << "k = " << NewTrial.k << endl;

   cout << "Лучший результат" << endl;
   cout << "z = " << BestTrial.z << endl;
   cout << "x = " << BestTrial.x << endl;
   cout << "k = " << BestTrial.k;
   return BestTrial;
}

int Solver::GetK() {
   Solver::h;
   h = BestTrial.k;
   return h;
}
int Solver::GetKcur() {
   Solver::hh;
   hh = NewTrial.k;
   return hh;
}

void Solver::FirstTrial(int i, double* minFunc, double* argmin, double* l, vector<Trial>&
trials) {
   trials[0].x = a; trials[0].z = obj_func(trials[0].x,i); trials[0].k = 1;
   trials[1].x = b; trials[1].z = obj_func(trials[1].x,i); trials[1].k = 2;
   *minFunc = trials[0].z, * argmin = trials[0].x, * l = trials[0].k;

   if (*minFunc > trials[1].z) {
      *minFunc = trials[1].z;
      *argmin = trials[1].x;
      *l = trials[1].k;
   }
}

void Solver::EstimateConstant(int *K, vector<Trial>& trials, double *m) {
   double M = 0, res;
   for (int i = 1; i < *K; i++) {
      res = fabs((trials[i].z - trials[i - 1].z) / (trials[i].x - trials[i - 1].x));
      if (res > M)
      M = res;
   }
   if (M == 0) *m = 1;
   else *m = M;
}

void  Solver::FindMaxR_BA_UPD(int*  K,  double*  minFunc,  vector<Trial>&  trials,  int*  t,
double* m) {
```

```
   double max = -10000;
   vector <double> R(*K);
   for (int i = 1; i < *K; i++) {
      R[i] = (trials[i].x - trials[i - 1].x) + (((trials[i].z - trials[i - 1].z) *
(trials[i].z - trials[i - 1].z)) / (*m * *m *r*r* (trials[i].x - trials[i - 1].x))) - 2 *
(trials[i].z + trials[i - 1].z - 2 * *minFunc)/ (*m*r);
      if (R[i] > max) {
         max = R[i];
         *t = i;
      }
   }
}

void SolverModif::FindMaxR_BA_Modif(int* K, double* minFunc, vector<Trial>& trials, int* t,
double* m, double* r_upd) {
   double max = -10000;
   vector <double> R(*K);
   vector <double> R_glob(*K);
   vector <double> R_loc(*K);
   for (int i = 1; i < *K; i++) {
      R_glob[i] = (trials[i].x - trials[i - 1].x) + (((trials[i].z - trials[i - 1].z) *
(trials[i].z - trials[i - 1].z)) / (*m * *m * r_glob * r_glob * (trials[i].x - trials[i -
1].x))) - 2 * (trials[i].z + trials[i - 1].z - 2 * *minFunc) / (*m * r_glob);
      R_loc[i] = (trials[i].x - trials[i - 1].x) + (((trials[i].z - trials[i - 1].z) *
(trials[i].z - trials[i - 1].z)) / (*m * *m * r_loc * r_loc * (trials[i].x - trials[i -
1].x))) - 2 * (trials[i].z + trials[i - 1].z - 2 * *minFunc) / (*m * r_loc);
      if (ro * R_loc[i] > R_glob[i]) {
         R[i] = ro * R_loc[i];
         *r_upd = r_loc;
      }
      else { R[i] = R_glob[i]; *r_upd = r_glob; }
      if (R[i] > max) {
         max = R[i];
         *t = i;
      }
   }
}

void SolverNew::FindMinR(int* K, vector<Trial>& trials, int* t) {
   double min = 10000;
   vector <double> R(*K);
   for (int i = 1; i < *K; i++) {
      R[i] = (trials[i].z + trials[i - 1].z)/(trials[i].x - trials[i - 1].x);
      if (R[i] < min) {
         min = R[i];
         *t = i;
      }
   }
}

void Solver::MakeNewTrial_BA(int i, int* K, vector<Trial>& trials, int* t, double* m) {
   NewTrial.x = (trials[*t].x + trials[*t - 1].x) / 2 - ((trials[*t].z - trials[*t - 1].z)
/ (2 * *m * r));
   NewTrial.z = obj_func(NewTrial.x,i);
   NewTrial.k = *K + 1;
   *K = *K + 1;
}
```

```
void SolverModif::MakeNewTrial(int i, int* K, vector<Trial>& trials, int* t, double* m,
double* r_upd) {
   NewTrial.x = (trials[*t].x + trials[*t - 1].x) / 2 - ((trials[*t].z - trials[*t - 1].z)
/ (2 * *m * *r_upd));
   NewTrial.z = obj_func(NewTrial.x, i);
   NewTrial.k = *K + 1;
   *K = *K + 1;
}

void SolverNew::MakeNewTrial(int i, int* K, vector<Trial>& trials, int* t) {
   NewTrial.x = (trials[*t].x + trials[*t - 1].x) / 2 - ((trials[*t].z - trials[*t - 1].z)
/ (trials[*t].z + trials[*t - 1].z)) * (trials[*t].x - trials[*t - 1].x) * 1 / (2 * r);
   NewTrial.z = obj_func(NewTrial.x, i);
   NewTrial.k = *K + 1;
   *K = *K + 1;
}

void Solver::UpdateOptimum(double* minFunc, double* argmin, double* l, vector<Trial>&
trials) {
   if (*minFunc > NewTrial.z) {
      *minFunc = NewTrial.z;
      *argmin = NewTrial.x;
      *l = NewTrial.k;
   }
}

void Solver::CheckStopCondition(bool *flag, vector<Trial>& trials, int* t) {
   if (trials[*t].x - trials[*t - 1].x <= eps * (b - a))
      *flag = 0;
   trials.insert(trials.begin() + *t, NewTrial);
}

void Solver::CheckStopConditionWithKnownOptPoint(int i, bool* flag, vector<Trial>& trials,
int* t) {
   if (abs(trials[*t].x - hpf[i]->GetOptimumPoint()[0]) <= eps * (b - a))
      *flag = 0;
   //if (abs(trials[*t].x - shf[i]->GetOptimumPoint()[0]) <= eps * (b - a))
      //*flag = 0;
   trials.insert(trials.begin() + *t, NewTrial);
}

void Solver::CheckStopConditionCurrentZ(int i, bool* flag, vector<Trial>& trials, int* t) {
   if ((trials[*t].z <= eps))
      *flag = 0;
   trials.insert(trials.begin() + *t, NewTrial);
}

void Solver::MakeBestTrial(double* minFunc, double* argmin, double* l) {
   BestTrial.z = *minFunc;
   BestTrial.x = *argmin;
   BestTrial.k = *l;
}
```

Полный код программы представлен здесь: https://github.com/alesyacurtis/multi-extremal-zero-finder